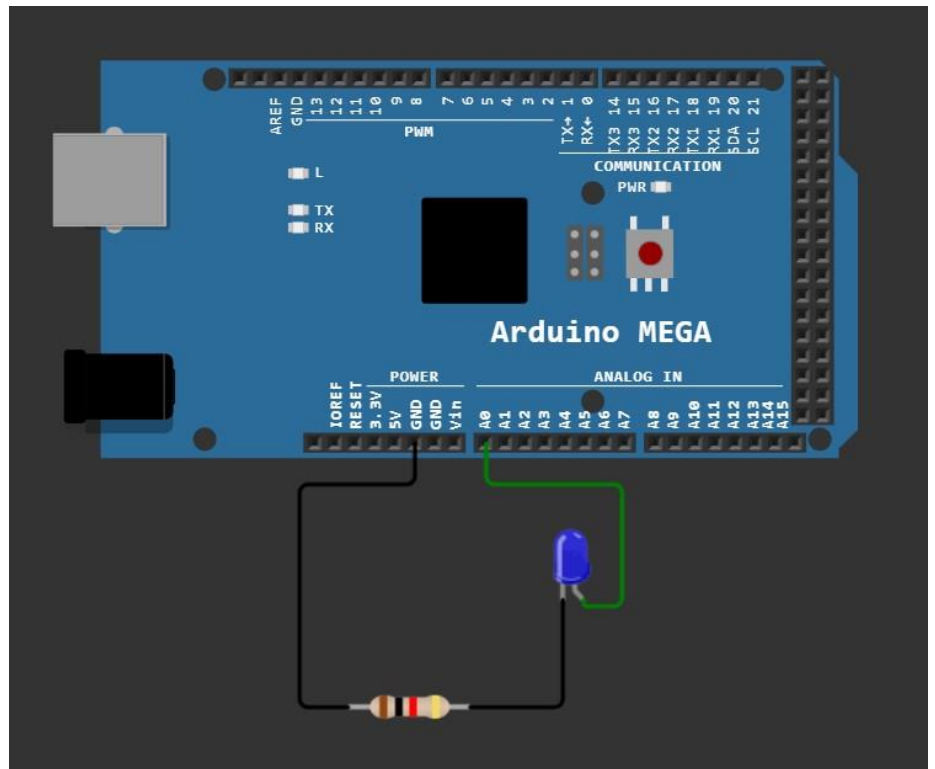


*Lecture Experiments of "Embedded Programming":-

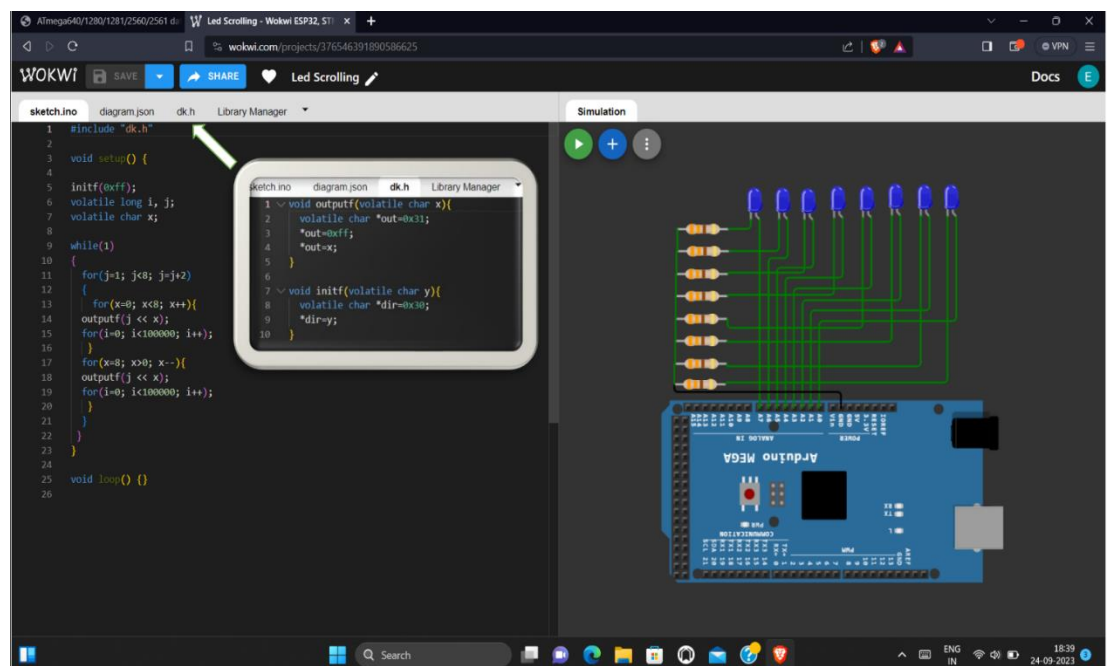
01. Led Blinking:-

```
void setup() {  
  
volatile char *dirf = 0x30; *dirf=0xff;  
volatile char *outf = 0x31;  
volatile long i;  
  
while(1)  
{  
    *outf=0x01;  
    for(i=0; i<100000; i++);  
    *outf=0x00;  
    for(i=0; i<100000; i++);  
}  
  
}  
  
void loop() {}
```



02. Led Scrolling:-

```
#include "dk.h"  
  
void setup() {  
    initf(0xff);  
    volatile long i, j;  
    volatile char x;  
  
    while(1)  
    {  
        for(j=1; j<8; j=j+2)  
        {  
            for(x=0; x<8; x++){  
                outputf(j << x);  
                for(i=0; i<100000; i++);  
            }  
            for(x=8; x>0; x--){  
                outputf(j << x);  
                for(i=0; i<100000; i++);  
            }  
        }  
    }  
}  
  
void loop() {}
```

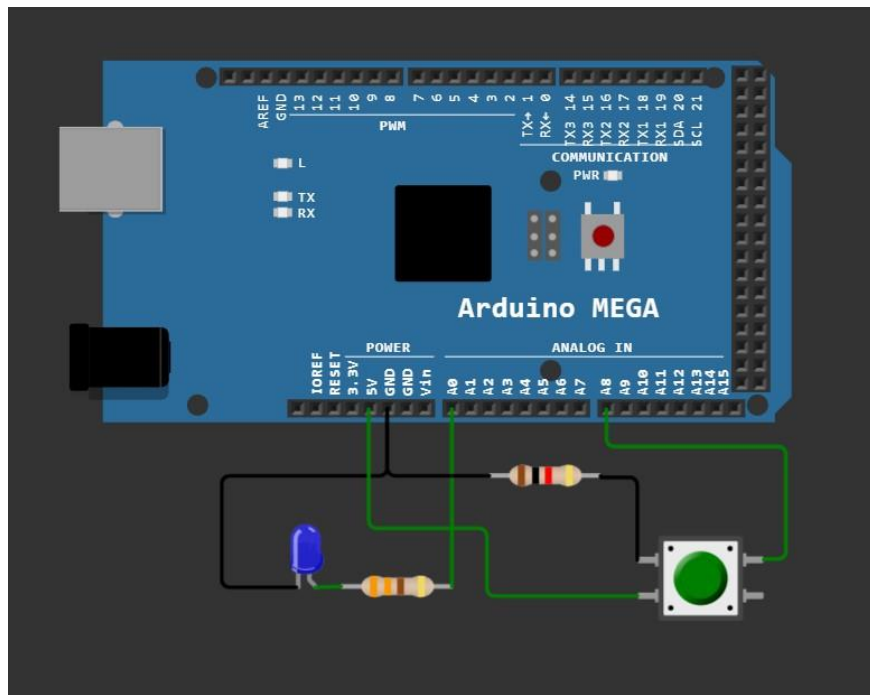


03. Led & Switch:-

```
void setup() {
  volatile char *dirf, *dirk, *outf, *ink;
  dirf=0x30; *dirf=0x01; // port f set as output
  dirk=0x107; *dirk=0x00; // port k set as input
  outf=0x31;
  ink=0x106;
  volatile char x;

  while(1){
    x=*ink;
    if((x & 1)==1) *outf=1;
    else *outf=0;
  }
}

void loop() {}
```



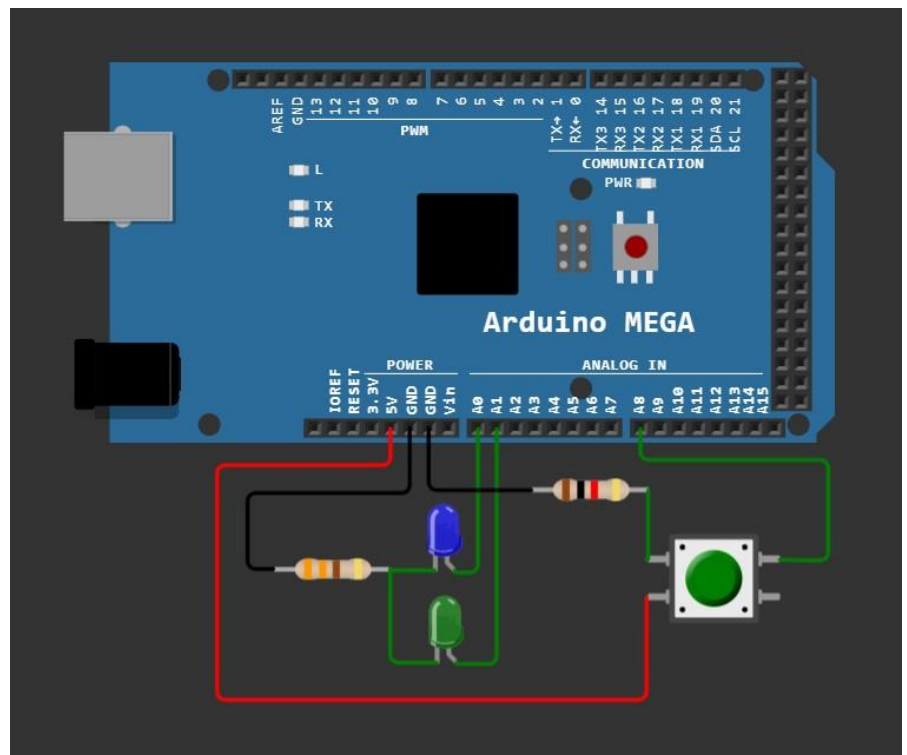
04.Two Leds & Switch:-

```
void setup() {
  volatile char *dirf, *dirk, *outf, *ink;
  dirf=0x30; *dirf=0x03; // pin 1 & 2 of port f set as output
  dirk=0x107; *dirk=0x00; // port k set as input
  outf=0x31;
  ink=0x106;
  volatile char x, i=0;
  volatile long j;

  while(1){
    x=*ink;
    if((x & 1)==1) i++;
    for(j=0; j<50000; j++);

    if(i%2) *outf=1;
    else *outf=2;
  }
}

void loop() {}
```

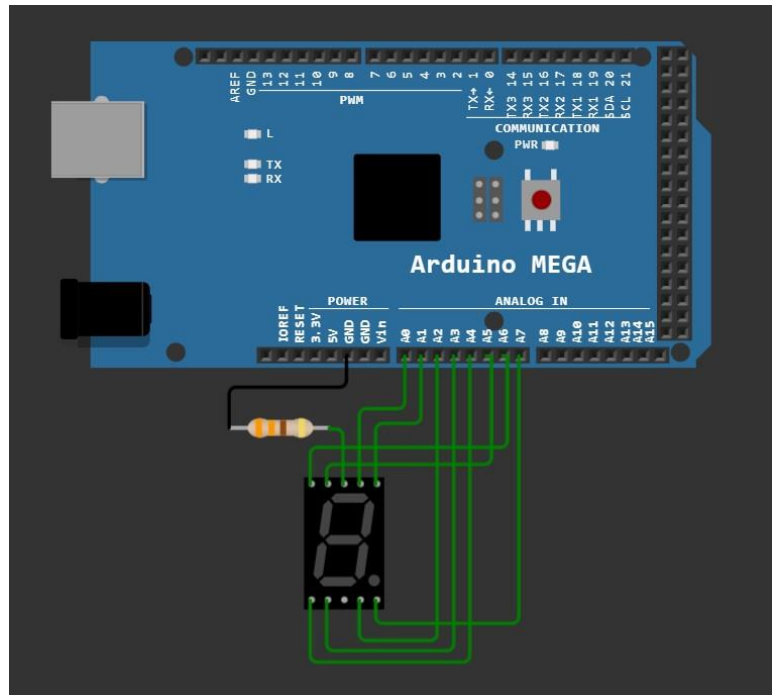


05. Seven Segment:-

```
void setup() {
  volatile char *dirf, *outf;
  volatile char a[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f, 0x6f};
  dirf=0x30;
  *dirf=0xff;
  outf=0x31;
  volatile long i, j;

  while(1) {
    for(i=0; i<10; i++)
    {
      *outf=a[i];
      for(j=0; j<500000; j++);
    }
  }

  void loop() {}
```

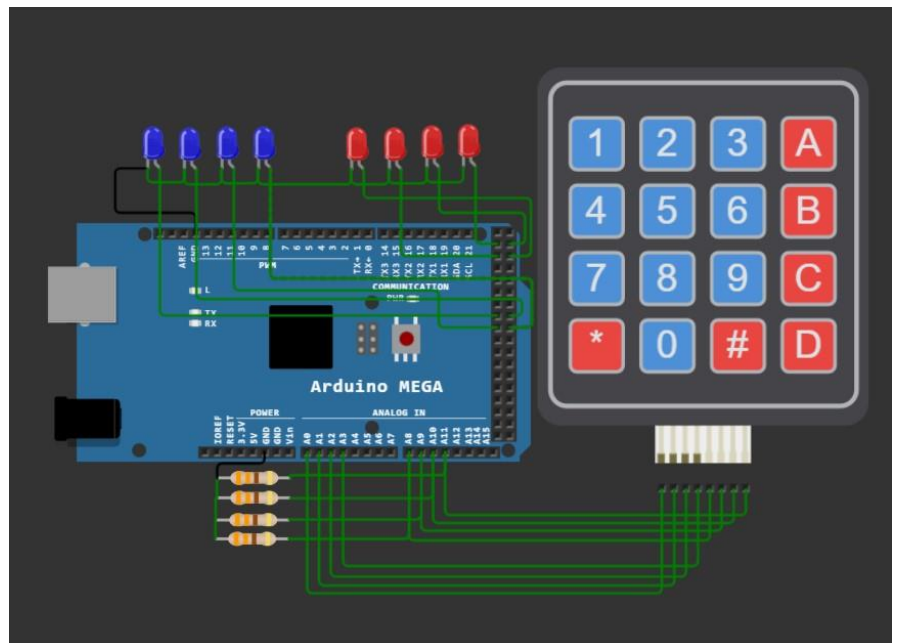


06. Keypad:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink, *dira, *outa, *dirc, *outc;
  volatile long i, j;

  dirf=0x30; *dirf=0xff;  outf=0x31;
  dirk=0x107; *dirk=0x00;  ink=0x106;
  dira=0x21; *dira=0xff;  outa=0x22;
  dirc=0x27; *dirc=0xff;  outc=0x28;

  while(1){
    for(i=0; i<4; i++)
    {
      *outf = 1 << i;
      *outa = *ink;
      if(*ink != 0){
        *outc = 1 << i;
        for(j=0; j<500000; j++);
        *outc = 0;
      }
    }
  }
  void loop() {}
```

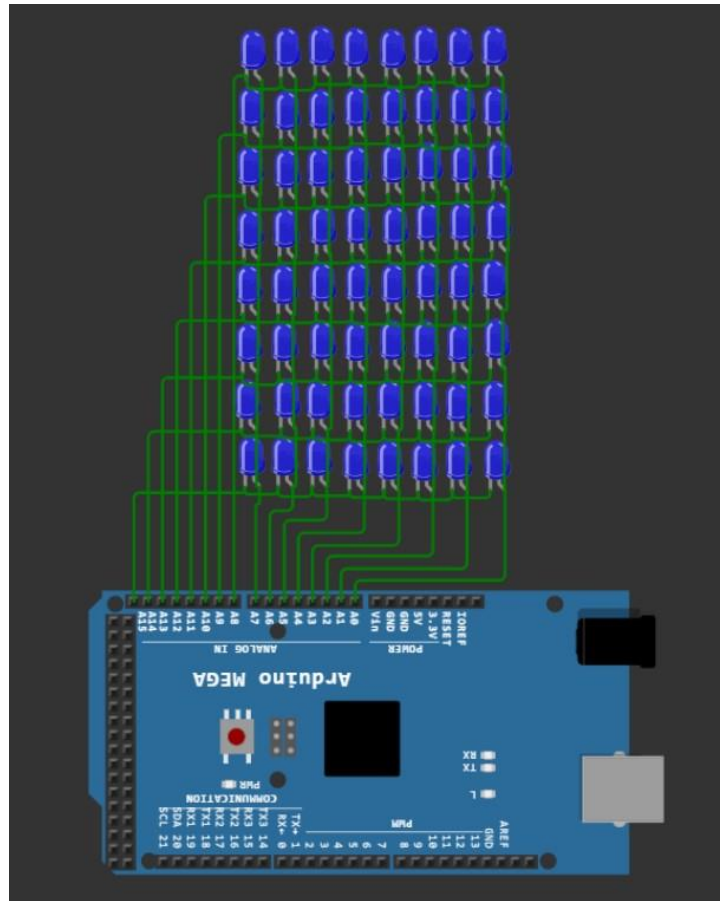


07. 8*8 Dot Matrix:-

```
void setup() {

volatile char *dirf, *dirk, *outf, *outk;
volatile long k;
char row, col;
dirf=0x30; *dirf=0xff;
dirk=0x107; *dirk=0xff;
outf=0x31; outk=0x108;
while(1)
{
for(row=0; row<8; row++){
*outk= ~ (1 << row);
for(col=0; col<8; col++){
*outf= 1 << col;
for(k=0; k<100000; k++);
}
*outk= 0xff;
*outf= 0;
}
}
}

void loop() {}
```



08. 8*8 Dot Matrix Display [ECEN]:-

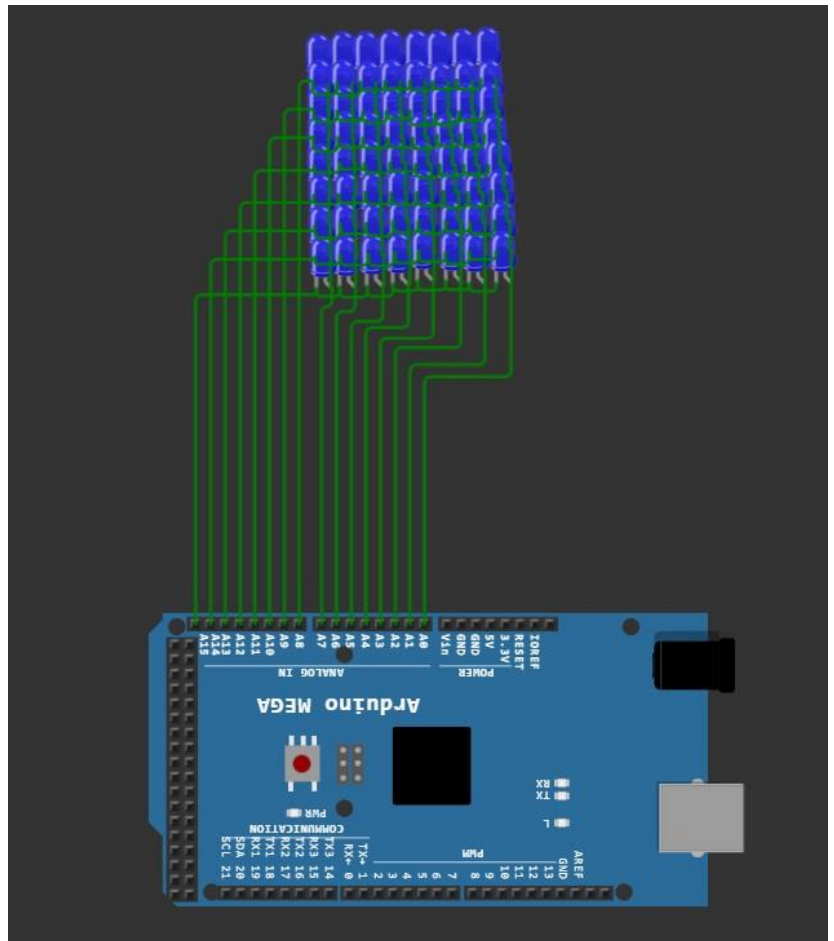
```
void setup() {

volatile char *dirf, *dirk, *outf, *outk;
volatile long a;
char i, count=0;

dirf=0x30; *dirf=0xff;
dirk=0x107; *dirk=0xff;
outf=0x31; outk=0x108;

volatile char ecen[][8] = {
{0b11111111,
0b11111111,
0b11000000,
0b11111111,
0b11111111,
0b11000000,
0b11111111,
0b11111111},

{0b00111110,
0b01111111,
```



```
0b11100011,  
0b11000000,  
0b11000000,  
0b11100011,  
0b01111111,  
0b00111110},
```

```
{0b11111111,  
0b11111111,  
0b11000000,  
0b11111111,  
0b11111111,  
0b11000000,  
0b11111111,  
0b11111111},
```

```
{0b11000011,  
0b11100011,  
0b11110011,  
0b11111011,  
0b11011111,  
0b11001111,  
0b11000111,  
0b11000011}};
```

```
while(1){  
for(a=0; a<20000; a++){  
for(i=0; i<8; i++)  
{  
*outk= ~(1 << i);  
*outf = ecen[count][i];  
*outk=0xff;  
*outf=0;  
}  
}
```

```
for(a=0; a<45000; a++);  
count++;  
if(count==4) count=0;
```

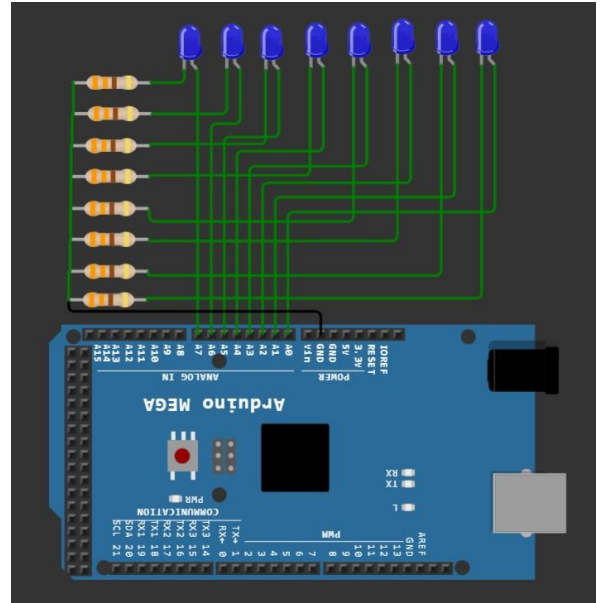
```
}  
}  
void loop() {}
```

* Embedded Programming Problem Sets:-

ESP - LED Problems – Set 1

1. Glow LSB LED only (bit 0):-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=1;  
}  
  
void loop() {}
```



2. Glow MSB LED only (bit 7):-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0x80;  
}  
  
void loop() {}
```

3. Glow LSB 2 LEDs only (bit 0 & bit 1):-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=3;  
}  
  
void loop() {}
```

4. Glow MSB 2 LEDs only (bit 7 & bit 6):-

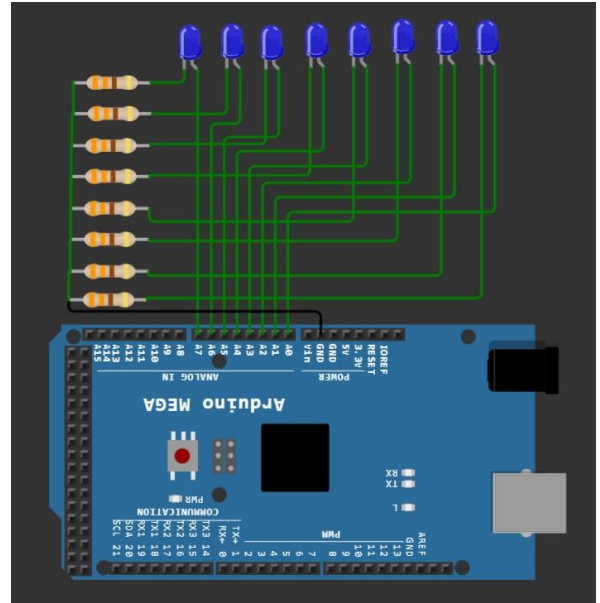
```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0xc0;  
}  
  
void loop() {}
```

5. Glow LSB 4 LEDs only (bit 0 to bit 3):-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0x0f;  
}  
  
void loop() {}
```

6. Glow MSB 4 LEDs only (bit 7 to bit 4):-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0xf0;  
}  
  
void loop() {}
```



7. Glow all 8 LEDs:-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0xff;  
}  
  
void loop() {}
```

8. Glow all ODD no LEDs:-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0xaa;  
}  
  
void loop() {}
```

9. Glow all EVEN no LEDs:-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0x55;  
}  
  
void loop() {}
```

10. Glow LEDs of bit7, bit4, bit3, bit1:-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30; *dirf=0xff;  
  outf=0x31;  
  
  *outf=0x99;  
}  
  
void loop() {}
```

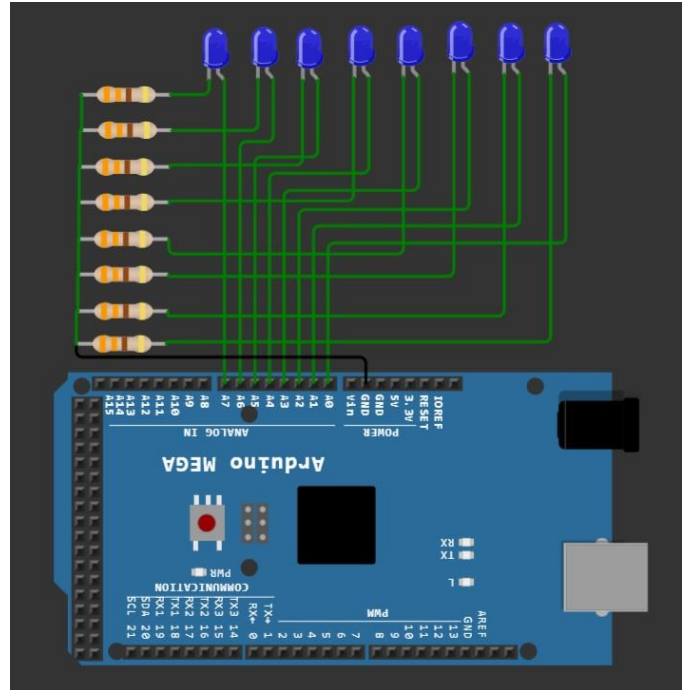

ESP - LED Problems – Set 2

1. Blink First LED (With Delay):-

```
void setup() {
  volatile char *dirf, *outf;
  volatile long i;
  dirf=0x30; *dirf=0xff;
  outf=0x31;

  while(1)
  {
    *outf=1;
    for(i=0; i<100000; i++);
    *outf=0;
    for(i=0; i<100000; i++);
  }
}

void loop() {}
```



2. Blink all the 8 LEDs (with delay):-

```
void setup() {
  volatile char *dirf, *outf;
  volatile long i;
  dirf=0x30; *dirf=0xff;
  outf=0x31;

  while(1)
  {
    *outf=0xff;
    for(i=0; i<100000; i++);
    *outf=0;
    for(i=0; i<100000; i++);
  }
}

void loop() {}
```

3. Glow the LEDs one by one (Do not off the LED):-

```
void setup() {
  volatile char *dirf, *outf, a;
  volatile long i;
  volatile short arr[]={2, 4, 8, 16, 32, 64, 128, 256};
  dirf=0x30; *dirf=0xff;
  outf=0x31;

  while(1)
  {
    for(a=0; a<8; a++)
    {
      *outf=arr[a]-1;
      for(i=0; i<100000; i++);
    }
    *outf=0;
  }
}

void loop() {}
```


4. Glow the LEDs one by one(at a time one LED should glow):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<8; a++)
{
*outf=1 << a;
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

5. Glow the EVEN LEDs one by one (Do not off the LED):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
volatile char arr[]={1, 4, 16, 64};
volatile short all=0;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
all += arr[a];
*outf=all;
for(i=0; i<100000; i++);
}
if(all >= 85) all=0;
*outf=0;
}
}

void loop() {}
```

6. Glow the ODD LEDs one by one (Do not off the LED):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
volatile char arr[]={1, 4, 16, 64};
volatile short all=0;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
all += (arr[a] * 2);
*outf=all;
for(i=0; i<100000; i++);
}
if(all >= (85 * 2)) all=0;
*outf=0;
}
}

void loop() {}
```

7. Glow 0 to 3 LEDs (one by one):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
*outf= 1 << a;
for(i=0; i<100000; i++);
}
}
}

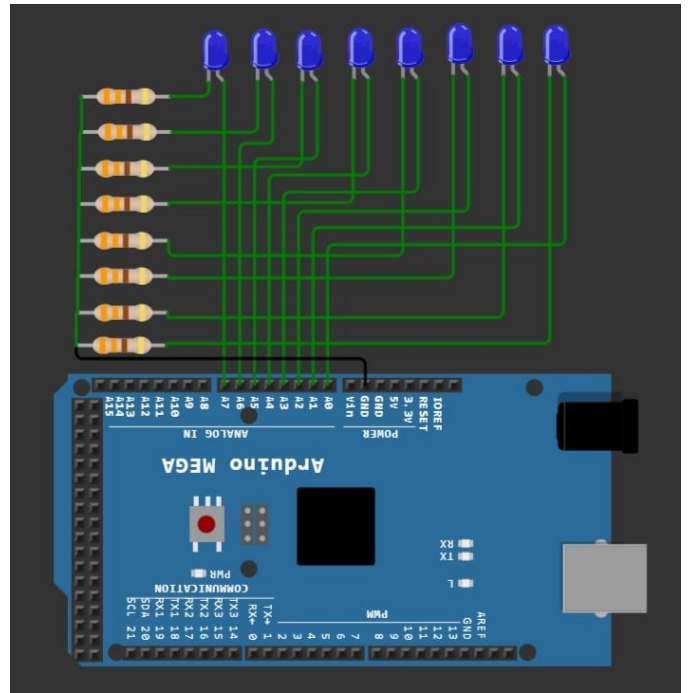
void loop() {}
```

8. Glow 4 to 7 LEDs (one by one):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
*outf= 16 << a;
for(i=0; i<100000; i++);
}
}
}

void loop() {}
```



9. Glow 3 to 0 LEDs (one by one):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
*outf= 8 >> a;
for(i=0; i<100000; i++);
}
}
}

void loop() {}
```

10. Glow 7 to 4 LEDs (one by one):-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
*outf= 128 >> a;
for(i=0; i<100000; i++);
}
}
}

void loop() {}
```

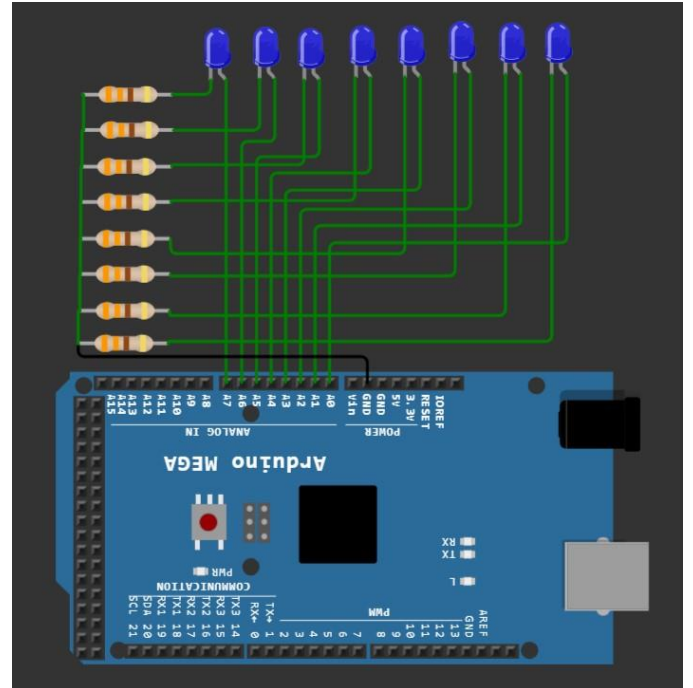
ESP - LED Problems – Set 3

1. Glow the LEDs in following order 3,7 delay – 2,6 delay – 1,5 delay – 0,4 delay:-

```
void setup() {
volatile char *dirf, *outf, a;
volatile char arr[]={136, 68, 34,17};
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
*outf=arr[a];
for(i=0; i<500000; i++);
}
}

void loop() {}
```



2. Glow the LEDs in following order 0,4 delay – 1,5 delay – 2,6 delay – 3,7 delay:-

```
void setup() {
volatile char *dirf, *outf, a;
volatile char arr[]={136, 68, 34,17};
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=3; a>=0; a--)
{
*outf=arr[a];
for(i=0; i<500000; i++);
}
}

void loop() {}
```

3. Glow 0 to 7 LEDs with delay and 7 to 0 LEDs with delay:-

```
void setup() {
volatile char *dirf, *outf, a;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1){
for(a=0; a<8; a++)
{
*outf= 1<<a;
for(i=0; i<100000; i++);
}

for(a=0; a<8; a++)
{
*outf= 128>>a;
for(i=0; i<100000; i++);
}
}

void loop() {}
```

4. Glow 0,2,1,3,4,6,5,7 LEDs with delay:-

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
volatile char arr[]={1,4,2,8,16,64,32,128};
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<8; a++)
{
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

5. Glow the LEDs in following order 0,1 delay – 6,7, delay – 2,3, delay – 4,5, delay

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
volatile char arr[]={3,192,12,48};
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<4; a++)
{
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

6. Glow the LEDs in following order 4,5 delay – 2,3, delay – 6,7, delay – 0,1, delay:-

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
volatile char arr[]={3,192,12,48};
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=3; a>=0; a--)
{
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

7. Glow the LEDs in following order 0,1,2 delay – 5,6,7, delay – 3,4, delay:-

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
volatile char arr[]={7,224,24};
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<3; a++)
{
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

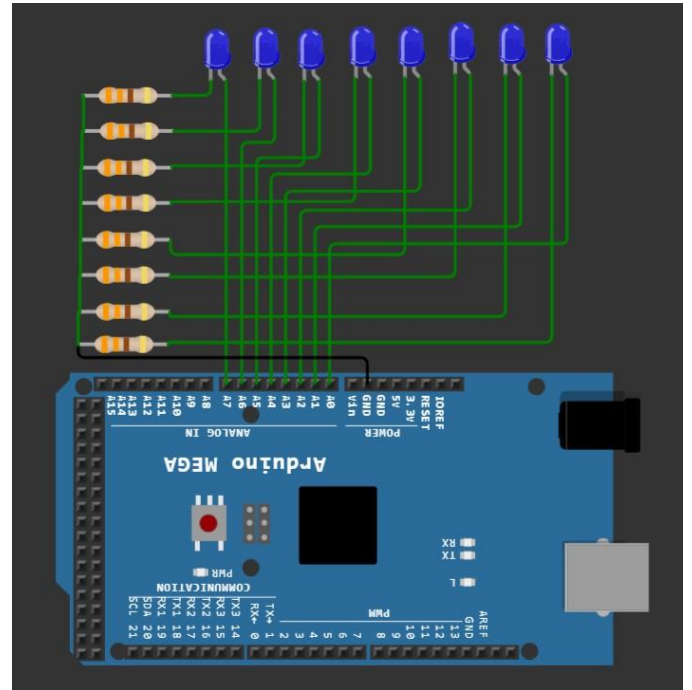
void loop() {}
```

8. Glow the LEDs in following order 3,4 delay – 5,6,7, delay – 0,1,2, delay:-

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
volatile char arr[]={7,224,24};
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=3; a>=0; a--)
{
*outf=arr[a];
for(i=0; i<200000; i++);
}
}

void loop() {}
```



9. Glow the LEDs in following order 0 then 0,1 then 0,1, 2..... then 0,1,2,3,4,5,6,7 with delay:-

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<8; a++)
{
*outf=(2 << a) -1;
for(i=0; i<200000; i++);
}
}

void loop() {}
```

10. Glow the LEDs in following order 7 then 7,6 then 7,6, 5..... then 7,6,5,4,3,2,1,0 with delay:-

```
void setup() {
volatile char *dirf, *outf, a, count=0;
volatile long i;
volatile char
arr[]={128,192,224,240,248,252,254,255};
dirf=0x30; *dirf=0xff;
outf=0x31;

while(1)
{
for(a=0; a<8; a++)
{
*outf=arr[a];
for(i=0; i<200000; i++);
}
}

void loop() {}
```

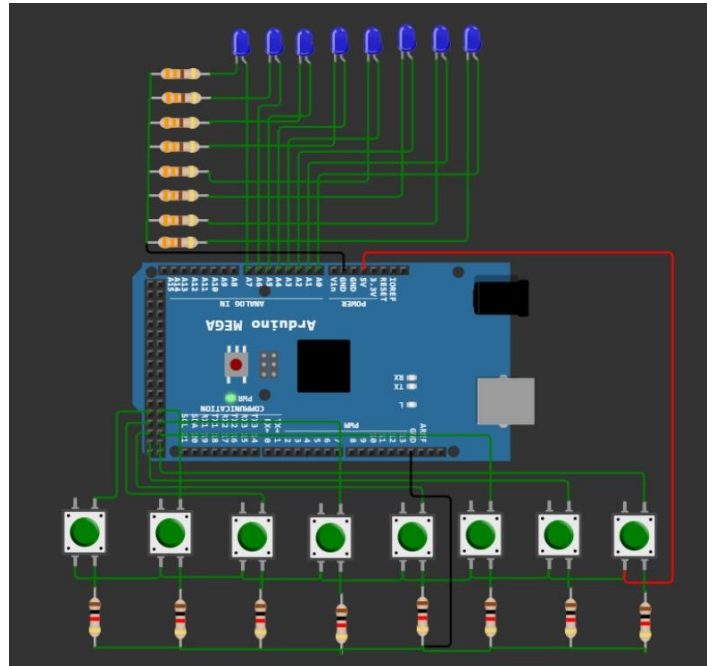
ESP - Switch Problems – Set 1

1. Press 0th switch glow 8 LEDs:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile char x;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0;
outf=0x31;
ina=0x20;

while(1){
  *outf=0;
  x=*ina;
  if(x==1){
    *outf=0xff;
    for(i=0; i<100000; i++);
  }
}

void loop() {}
```



2. Press 2nd switch blink 8 LEDs:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile char x;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0;
outf=0x31;
ina=0x20;

while(1){
  x=*ina;
  if(x==4){
    while(1){
      *outf=0xff;
      for(i=0; i<100000; i++);
      *outf=0;
      for(i=0; i<100000; i++);
    }
  }
}

void loop() {}
```

3. Press 3rd switch glow 0th LED, delay,1st LED, delay ...:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile char x, a;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0;
outf=0x31;
ina=0x20;

while(1){
  *outf=0;
  x=*ina;
  if(x==8){
    for(a=0; a<8; a++){
      *outf= 1 << a;
      for(i=0; i<100000; i++);
    }
  }
}

void loop() {}
```

4. Press any switch glow 7th LED, delay,6th LED, delay ...:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile char x, a;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0;
outf=0x31;
ina=0x20;

while(1){
    *outf=0;
    x=*ina;
    if(x){
        for(a=0; a<8; a++){
            *outf= 128 >> a;
            for(i=0; i<1000000; i++){
            }
        }
    }
}

void loop() {}
```

5. Press 0th switch glow even LEDs:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile char x;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0;
outf=0x31;
ina=0x20;

while(1){
    *outf=0;
    x=*ina;
    if(x==1){
        *outf=0x55;
        for(i=0; i<5000000; i++){
        }
    }
}

void loop() {}
```

6. Press 7th switch glow odd LEDs:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile short x;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){
    *outf=0;
    x=*ina;
    if((x & 128) == 128){
        *outf=0xaa;
        for(i=0; i<5000000; i++){
        }
    }
}

void loop() {}
```

7. Press 0th switch 0th LED should glow Press 1st switch 1st LED should glowup to 7th switch:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile long i;
volatile long x;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){

    *outf=*ina;

}

}

void loop() {}
```


8. Press 0th switch 7th LED should glow

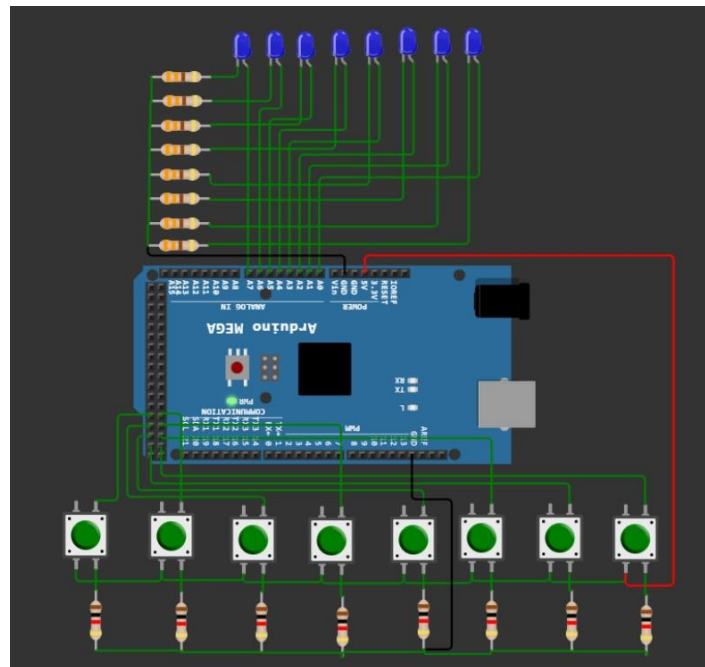
Press 1st switch 6th LED should glow

.....up to 7th switch:-

```
void setup() {  
  volatile char *dirf, *outf, *dira, *ina;  
  volatile short x;  
  dirf=0x30; *dirf=0xff;  
  dira=0x21; *dira=0x00;  
  outf=0x31;  
  ina=0x20;  
}
```

```
while(1){  
  x=*ina;  
  if(x==1) *outf=128;  
  else if(x==2) *outf=64;  
  else if(x==4) *outf=32;  
  else if(x==8) *outf=16;  
  else if(x==16) *outf=8;  
  else if(x==32) *outf=4;  
  else if(x==64) *outf=2;  
  else if((x & 128)==128) *outf=1;  
  else *outf=0;  
}
```

```
void loop() {}
```



9. Press any switch 0th LED ON, OFF 1st LED ON, OFF.....:-

```
void setup() {  
  volatile char *dirf, *outf, *dira, *ina;  
  volatile long i;  
  volatile char a, x;  
  dirf=0x30; *dirf=0xff;  
  dira=0x21; *dira=0x00;  
  outf=0x31;  
  ina=0x20;  
}
```

```
while(1){  
  *outf=0;  
  x=*ina;  
  if(x)  
    for(a=0; a<8; a++){  
      *outf= 1 << a;  
      for(i=0; i<100000; i++);  
    }  
}
```

```
void loop() {}
```

10. Press 1st switch At a time glow 0th LED,3rd LED, 4th LED,7th LED:-

```
void setup() {  
  volatile char *dirf, *outf, *dira, *ina;  
  volatile long i;  
  volatile char a, x;  
  dirf=0x30; *dirf=0xff;  
  dira=0x21; *dira=0x00;  
  outf=0x31;  
  ina=0x20;  
}
```

```
while(1){  
  *outf=0;  
  x=*ina;  
  if(x){  
    *outf=153;  
    for(i=0; i<500000; i++);  
  }  
}
```

```
void loop() {}
```

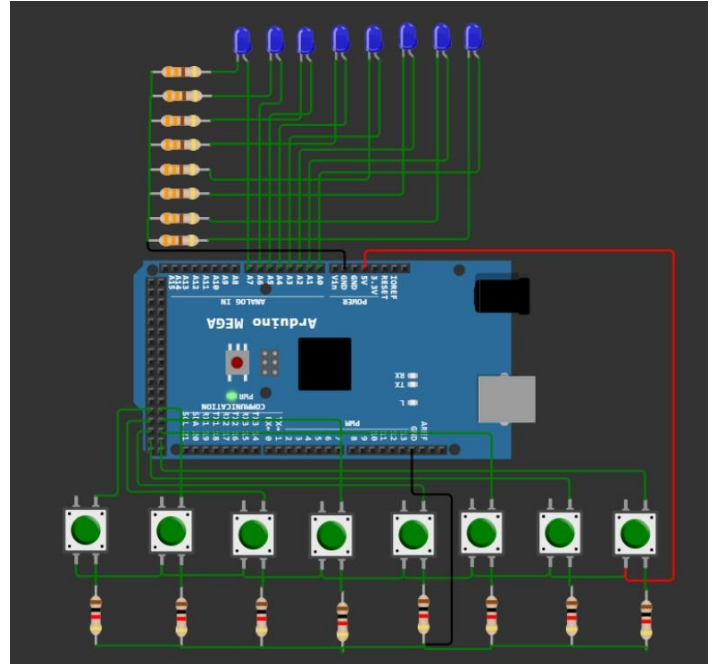
ESP - Switch Problems – Set 2

1. Press 2nd switch first glow 1st LED, delay, 2nd LED, delay, 5th LED, delay, 6th LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={2,4,32,64}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;
```

```
while(1){
*outf=0;
x=*ina;
if(x==4)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}
```

```
void loop() {}
```



2. Press 3rd switch first glow 3rd LED, delay, 1st LED, delay, 7th LED, delay, 4th LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={8,1,128,16}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;
```

```
while(1){
*outf=0;
x=*ina;
if(x==8)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}
```

```
void loop() {}
```

3. Press 2nd switch glow 0th LED, delay, 1 st LED, delay, 5th LED, delay, 6th LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={1,2,32,64}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;
```

```
while(1){
*outf=0;
x=*ina;
if(x==4)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}
```

```
void loop() {}
```

4. Press 7th switch first glow 0th LED, delay, 1st LED, Delay, 4th LED, delay, 7th LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={1,2,16,128}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){
*outf=0;
x=*ina;
if((x & 128)==128)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

5. Press 3rd switch first glow 7th LED, delay, 6th LED, delay, 3rd LED, delay, 2nd LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={128,64,8,4}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){
*outf=0;
x=*ina;
if(x==8)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

6. Press 2nd switch first glow 3rd LED, delay, 5th LED, delay, 7th LED, delay, 6th LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={8,32,128,64}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){
*outf=0;
x=*ina;
if(x==4)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

void loop() {}
```

7. Press 7th switch first glow 5th LED, delay, 3rd LED, delay, 0th LED, delay, 1st LED:-

```
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile char arr[]={32,8,1,2}, a;
volatile unsigned short x;
volatile long i;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){
*outf=0;
x=*ina;
if((x & 128) ==128)
for(a=0; a<4; a++){
*outf=arr[a];
for(i=0; i<500000; i++);
}
}
}

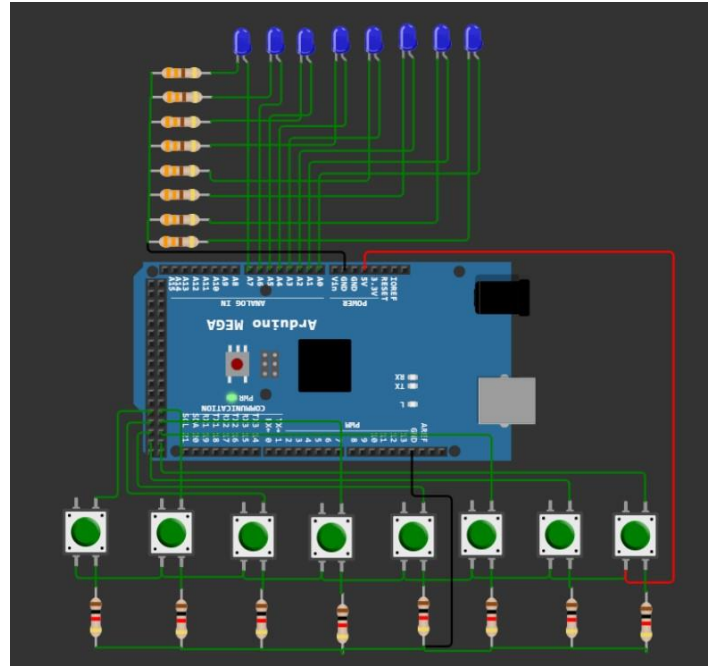
void loop() {}
```

8. Press 1st switch & Press 7th switch Together, Glow 5th LED for 30 seconds:-

```
void delays(char y);
void setup() {
volatile char *dirf, *outf, *dira, *ina;
volatile unsigned short x;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31;
ina=0x20;

while(1){
*outf=0; x=*ina;
if(x==130){
*outf=32; delays(30);
} } }
void loop() {}

void delays(char y){ // delay(10); = 10 second delay;
volatile long i, j;
for(i=0; i<y; i++)
for(j=0; j<430000; j++);
}
```



9. Press 1st switch & Press 4th switch & Press 6th switch & Press 2nd switch One by One, Glow 7th LED for 10 seconds:-

```
void delays(char y);
void setup() {
volatile char *dirf, *outf, *dira, *ina;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31; ina=0x20;

L1:
while(1) if(*ina==2)
while(1) if(*ina==16)
while(1) if(*ina==64)
while(1) if(*ina==4)
{
*outf=128; delays(10); *outf=0;
goto L1;
}
}
void loop() {}

void delays(char y){ // delay(10); = 10 second delay;
volatile long i, j;
for(i=0; i<y; i++)
for(j=0; j<430000; j++);
}
```

10. Press 1st switch & Press 4th switch & Press 6th switch & Press 5th switch One by One, Glow 5th LED; Press 4th switch & Press 6th switch & Press 5th switch & Press 3th switch & One by One, OFF 5th LED:-

```
void delays(char y);
void setup() {
volatile char *dirf, *outf, *dira, *ina;
dirf=0x30; *dirf=0xff;
dira=0x21; *dira=0x00;
outf=0x31; ina=0x20;

L1: while(1) {
if(*ina==2){
while(1) if(*ina==16)
while(1) if(*ina==64)
while(1) if(*ina==32) {*outf=32; goto L1;}}

if(*ina==16){
while(1) if(*ina==64)
while(1) if(*ina==32)
while(1) if(*ina==8) {*outf=0; goto L1;}}
} }
void loop() {}
```


04 Display odd numbers with delay:-

```
void setup() {
  volatile char *dirf, *outf;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};

  volatile char a;
  volatile long i;
  dirf=0x30;
  *dirf=0xff;
  outf=0x31;

  while(1) {
    for(a=1; a<10; a+=2)
    {
      *outf=arr[a];
      for(i=0; i<300000; i++);
    }
  }

  void loop() {}
```

05 Display even numbers with delay:-

```
void setup() {
  volatile char *dirf, *outf;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};

  volatile char a;
  volatile long i;
  dirf=0x30;
  *dirf=0xff;
  outf=0x31;

  while(1) {
    for(a=0; a<10; a+=2)
    {
      *outf=arr[a];
      for(i=0; i<300000; i++);
    }
  }

  void loop() {}
```

06 Display the possible alphabets (A to Z):-

```
void setup() {
  volatile char *dirf, *outf;
  volatile char arr[]={0x77, 0x7c,0x58,0x5e,
                      0x79,0x71,0x40,0x76,0x06,0x0e,0x40,0x38,
                      0x40,0x80,0x5c,0x73,0x67,0x50,0x6d,0x40,
                      0x1c,0x40,0x80,0x40,0x80,0x40};
  volatile char a;
  volatile long i;
  dirf=0x30;
  *dirf=0xff;
  outf=0x31;

  for(a=0; a<26; a++)
  {
    *outf=arr[a];
    for(i=0; i<500000; i++);
  }
  *outf=0;
}

void loop() {}
```

07 Display numbers in following order: 0,2,4,6,8,9,7,5,3,1:-

```
void setup() {
  volatile char *dirf, *outf;
  volatile char arr[]={0x3f,0x5b,0x66,0x7d,0x7f,
                      0x6f,0x07,0x6d,0x4f, 0x06};

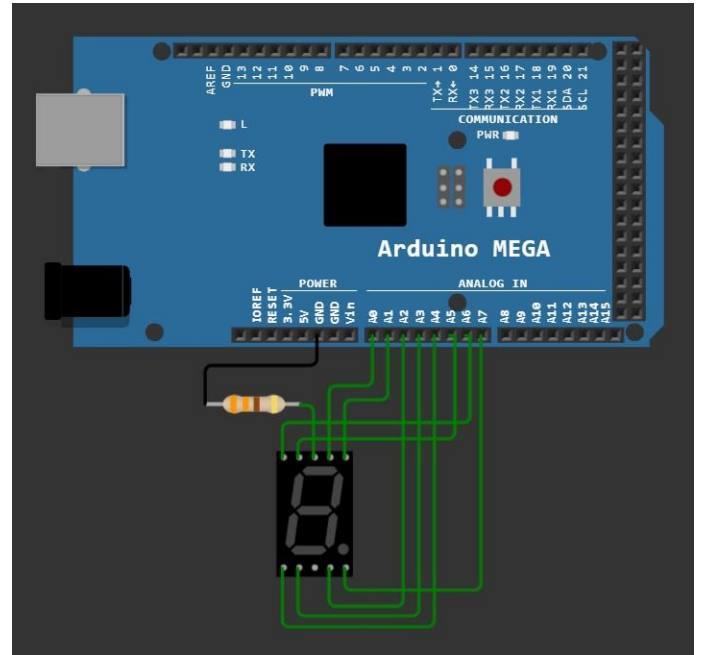
  volatile char a;
  volatile long i;
  dirf=0x30;
  *dirf=0xff;
  outf=0x31;

  while(1) {
    for(a=0; a<10; a++)
    {
      *outf=arr[a];
      for(i=0; i<300000; i++);
    }
  }

  void loop() {}
```

08 Display the single digit prime numbers:-

```
void setup() {  
  volatile char *dirf, *outf;  
  volatile char arr[]={0x5b,0x4f,0x6d,0x07};  
  volatile long i;  
  volatile char a;  
  dirf=0x30;  
  *dirf=0xff;  
  outf=0x31;  
  
  while(1) {  
    for(a=0; a<4; a++)  
    {  
      *outf=arr[a];  
      for(i=0; i<300000; i++);  
    }  
  }  
}  
  
void loop() {}
```



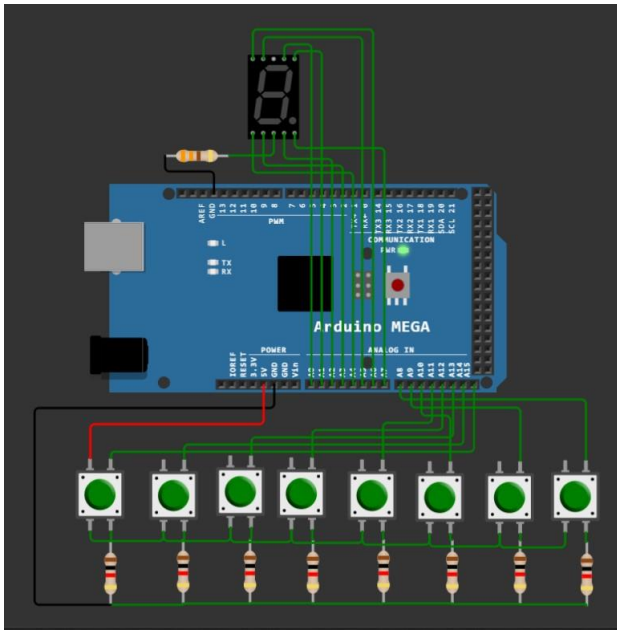
09 Display last digit of a number. Assume any number.:-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30;  
  *dirf=0xff;  
  outf=0x31;  
  
  *outf=0x6d;  
}  
  
void loop() {}
```

10 Display first digit of a number. Assume any number.:-

```
void setup() {  
  volatile char *dirf, *outf;  
  dirf=0x30;  
  *dirf=0xff;  
  outf=0x31;  
  
  *outf=0x5b;  
}  
  
void loop() {}
```

ESP – 7 Segment Problems – Set 2



01 Press any switch & display that number.

Ex: 1 -> 1, 2 ->2, ... 8 -> 8:-

```
void delays(char y);

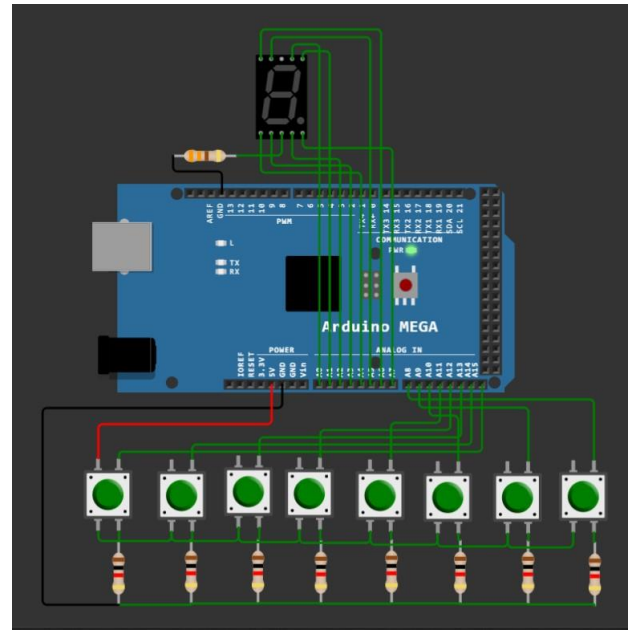
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char arr[]={0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f};

  volatile char a;
  volatile unsigned short x;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    x=*ink;
    if(x)
    for(a=0; a<8; a++)
    {
      *outf=arr[a]; delays(1);
      *outf=0x40; delays(1);
      *outf=arr[a]; delays(1);
    }
    *out=0;
  }
}

void loop() {}

void delays(char y){ // delay(10); = 10 second delay;
  volatile long i, j;
  for(i=0; i<y; i++)
    for(j=0; j<430000; j++);
}
```



02 Press any switch & display the reverse

order Ex: 1 -> 8, 2-> 7, 8 -> 1:-

```
void delays(char y);

void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char arr[]={0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f};

  volatile char a, b;
  volatile unsigned short x;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    x=*ink;
    if(x)
    for(a=0, b=7; a<8; a++, b--){
      *outf=arr[a]; delays(1);
      *outf=0x40; delays(1);
      *outf=arr[b]; delays(1);
    }
    *outf=0;
  }
}

void loop() {}

void delays(char y){ // delay(10); = 10 second delay;
  volatile long i, j;
  for(i=0; i<y; i++)
    for(j=0; j<430000; j++);
}
```


03 Press any switch & Display should start from that number till nine and restart from that number till 9. Continuous loop:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
0x6d,0x7d,0x07,0x7f, 0x6f};
  volatile char a, b;
  volatile long i;
  volatile unsigned short x;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    x=*ink;
    if(x==1) a=0;
    else if(x==1) a=0;    else if(x==2) a=1;
    else if(x==4) a=2;    else if(x==8) a=3;
    else if(x==16) a=4;   else if(x==32) a=5;
    else if(x==64) a=6;   else if((x & 128)==128) a=7;

    if(x) while(1){
      for(b=a; b<10; b++){
        *outf=arr[b];
        for(i=0; i<200000; i++);
      }
    }
  }
  void loop() {}
```

05 Display A – F in a loop:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char arr[]={0x77, 0x7c,0x58,0x5e,0x79,0x71};
  volatile char a;
  volatile long i;
  dirf=0x30; *dirf=0xff; outf=0x31;
  // dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    for(a=0; a<6; a++){
      *outf=arr[a];
      for(i=0; i<300000; i++);
    }
  }
  void loop() {}
```

04 Press switch 7 & display from 0 to 9 and 9 to 0 one time:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char
arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,
0x6f};
  volatile char a;
  volatile long i;
  volatile unsigned short x;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    x=*ink;
    *outf=0;
    if((x & 128)==128){
      for(a=0; a<10; a++){
        *outf=arr[a];
        for(i=0; i<300000; i++);
      }
      for(a--; a>=0; a--){
        *outf=arr[a];
        for(i=0; i<300000; i++);
      }
    }
  }
  void loop() {}
```

06 Initially display 0. When switch 1 is pressed every time increment by 1:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                        0x6d,0x7d,0x07,0x7f, 0x6f};
  volatile char count=0;
  volatile long i;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  *outf=arr[count];
  while(1) {
    if(*ink==2){
      count++;
      *outf=arr[count];
      for(i=0; i<100000; i++);
    }
    if(count==9) {count=0; count--;}
  }
  void loop() {}
```

07 Initially display 9. When switch 2 is pressed every time decrement by 1:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};
  volatile char count=9;
  volatile long i;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  *outf=arr[count];
  while(1) {
    if(*ink==4){
      count--;
      *outf=arr[count];
      for(i=0; i<100000; i++);
    }
    if(count==0) count=10;
  }
}
void loop() {}
```

08 Initially display 0 When switch 1 is pressed start displaying 0 to 9 Continuously. When switch 2 is pressed stop display:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char a, arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                        0x6d,0x7d,0x07,0x7f, 0x6f};
  volatile long i;
  volatile char flag=0;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    *outf=0x3f; if(*ink==2)
    while(1){
      for(a=0; a<10; a++){
        *outf=arr[a]; for(i=0; i<300000; i++)
        if(*ink==4) flag=1;
        if(flag==1) break;
      }
      if(flag==1){flag=0; break;}
    }
  }
}
void loop() {}
```

09 Initially display 9 When switch 1 is pressed start displaying 9 to 0 Continuously. When Switch 2 is pressed stop display:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char a, arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                        0x6d,0x7d,0x07,0x7f, 0x6f};
  volatile long i;
  volatile char flag=0;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;

  while(1) {
    *outf=0x6f;
    if(*ink==2)
    while(1){
      for(a=9; a>=0; a--){
        *outf=arr[a];
        for(i=0; i<300000; i++)
        if(*ink==4) flag=1;
        if(flag==1) break;
      }
      if(flag==1){flag=0; break;}
    }
  }
}
void loop() {}
```

10 Get a 4 digit number from user (Press the switch Sequentially (Example 1256) Add the sum of Them the sum is more than 1 digit then add them again and show the sum on display:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *ink;
  volatile char a, arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                        0x6d,0x7d,0x07,0x7f, 0x6f};
  volatile char count=0, x, sum=0;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0; ink=0x106;
  volatile long i;

  while(1){
    x=*ink;
    if(x==1) a=0; else if(x==2) a=1; else if(x==4) a=2;
    else if(x==8) a=3; else if(x==16) a=4; else if(x==32) a=5;
    else if(x==64) a=6; else if((x & 128)==128) a=7;

    if(x){
      sum+=a; count++;
      for(i=0; i<90000; i++); }
    if(count==4) break;
  }
  while(sum>9){ sum=(sum/10) + (sum%10); }
  *outf=arr[sum];
}
void loop() {}
```

ESP – 7 Segment Problems – Set 3

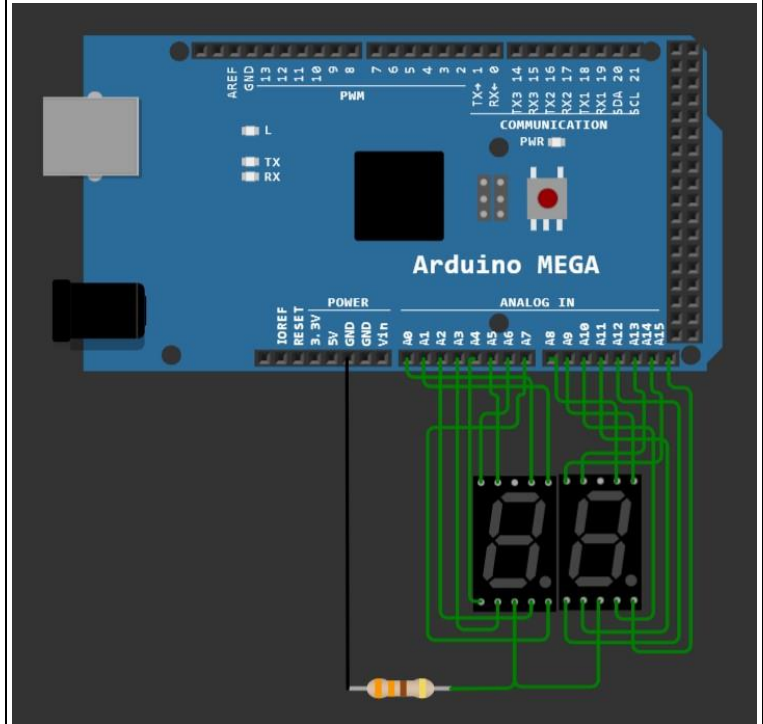
01 Display 00 to 99 in loop with delay:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  volatile long i;
  volatile char a, b;

  while(1) {
    for(a=0; a<10; a++)
      for(b=0; b<10; b++)
      {
        *outf=arr[a];
        *outk=arr[b];
        for(i=0; i<300000; i++);
      }
  }

  void loop() {}
```

Connect TWO 7-Segment Displays to 2 individual ports



02 Display 0.0 to 9.9 in loop with delay:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  volatile long i;
  volatile char a, b;

  while(1) {
    for(a=0; a<10; a++)
      for(b=0; b<10; b++)
      {
        *outf=arr[a] + 128;
        *outk=arr[b];
        for(i=0; i<300000; i++);
      }
  }

  void loop() {}
```

03 Display 99 to 00 in loop with delay:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  volatile long i;
  volatile char a, b;

  while(1) {
    for(a=9; a>=0; a--)
      for(b=9; b>=0; b--)
      {
        *outf=arr[a];
        *outk=arr[b];
        for(i=0; i<300000; i++);
      }
  }

  void loop() {}
```

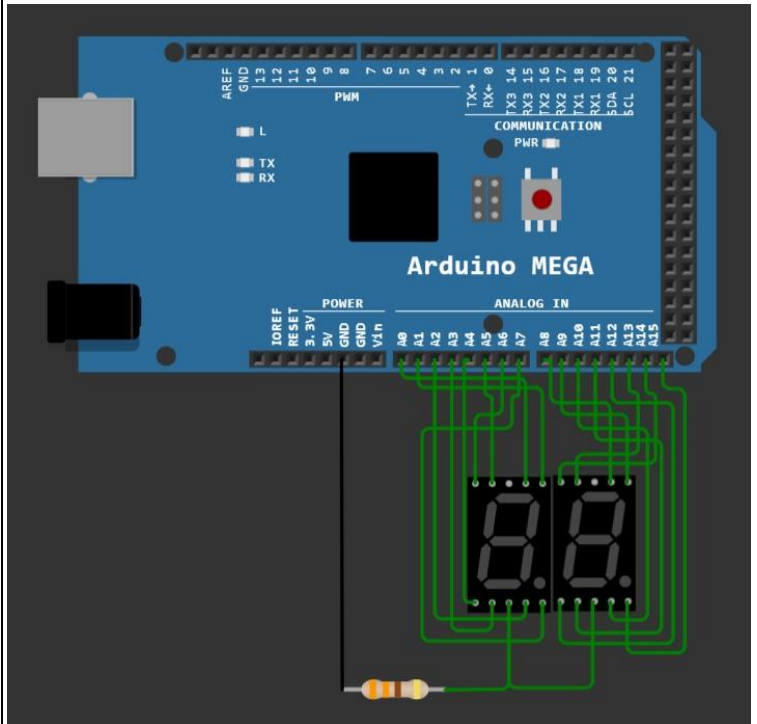
04 Display 9.9 to 0.0 in loop with delay:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                       0x6d,0x7d,0x07,0x7f, 0x6f};
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  volatile long i;
  volatile char a, b;

  while(1) {
    for(a=9; a>=0; a--)
      for(b=9; b>=0; b--)
      {
        *outf=arr[a] + 128;
        *outk=arr[b];
        for(i=0; i<300000; i++);
      }
  }

  void loop() {}
```

Connect TWO 7-Segment Displays to 2 individual ports



05 Display 0000 to 9999 in loop with delay:-

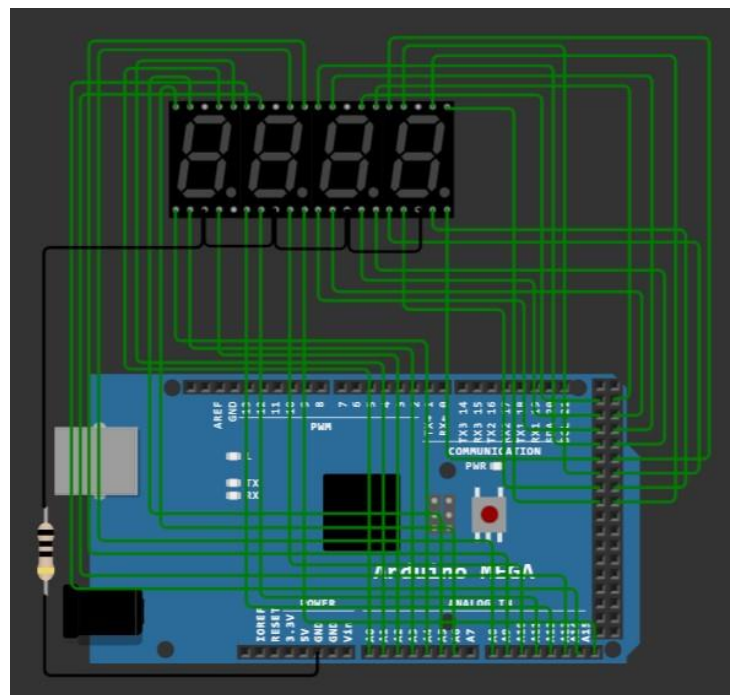
```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk, *dira,
               *outa, *dirc, *outc;
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                       0x6d,0x7d,0x07,0x7f, 0x6f};
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  dira=0x21; *dira=0xff; outa=0x22;
  dirc=0x27; *dirc=0xff; outc=0x28;

  volatile long i;
  volatile char a, b, c, d;

  while(1) {
    for(a=0; a<10; a++)
      for(b=0; b<10; b++)
        for(c=0; c<10; c++)
          for(d=0; d<10; d++)
          {
            *outf=arr[a];
            *outk=arr[b];
            *outa=arr[c];
            *outc=arr[d];
            for(i=0; i<50000; i++);
          }
  }

  void loop() {}
```

Connect FOUR 7-Segment Displays to 4 individual ports



06 Display 0.000 to 999.9 in loop with delay:-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk, *dira,
               *outa, *dirc, *outc;

  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  dira=0x21; *dira=0xff; outa=0x22;
  dirc=0x27; *dirc=0xff; outc=0x28;

  volatile long i;
  volatile char a, b, c, d;
  volatile char count=0, m, n, o, p;

  while(1) {
    if(count==0){m=128; n=0; o=0; p=0;}
    if(count==1){m=0; n=128; o=0; p=0;}
    if(count==2){m=0; n=0; o=128; p=0;}
    if(count==3){m=0; n=0; o=0; p=128;}

    for(a=0; a<10; a++)
    for(b=0; b<10; b++)
    for(c=0; c<10; c++)
    for(d=0; d<10; d++)
    {
      *outf=arr[a] + m;
      *outk=arr[b] + n;
      *outa=arr[c] + o;
      *outc=arr[d] + p;
      for(i=0; i<1000; i++);
    }
    count++;
    if(count==4) goto end;
  }
  end:
  *outf=0; *outk=0; *outa=0; *outc=0;
}

void loop() {}
```

07 Display 0 to 9999 in loop with delay. Do not display the prefix 0 :-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk, *dira,
               *outa, *dirc, *outc;

  volatile char arr[]={0x00, 0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  dira=0x21; *dira=0xff; outa=0x22;
  dirc=0x27; *dirc=0xff; outc=0x28;

  volatile long i, count=0;
  volatile char a, b, c, d;
  volatile char m, n, o, p;

  while(1) {
    for(a=1; a<=10; a++)
    for(b=1; b<=10; b++)
    for(c=1; c<=10; c++)
    for(d=1; d<=10; d++)
    {
      if(count>=1000) {m=1; n=1; o=1; p=1;}
      else if(count>=100) {m=0; n=1; o=1; p=1;}
      else if(count>=10) {m=0; n=0; o=1; p=1;}
      else if(count>=1) {m=0; n=0; o=0; p=1;}

      *outf=arr[a*m];
      *outk=arr[b*n];
      *outa=arr[c*o];
      *outc=arr[d*p];
      for(i=0; i<300000; i++);

      count++;
      if(count>10000) goto end;
    }
  }
  end:
  *outf=0; *outk=0; *outa=0; *outc=0;
}

void loop() {}
```

08 Display 9999 to 0 in loop with delay. Do not display the prefix 0 :-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *outc;

  volatile char arr[]={0x00, 0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  dira=0x21; *dira=0xff; outa=0x22;
  dirc=0x27; *dirc=0xff; outc=0x28;

  volatile long i, count=9999;
  volatile char a, b, c, d;
  volatile char m, n, o, p;

  while(1) {
    for(a=10; a>0; a--)
    for(b=10; b>0; b--)
    for(c=10; c>0; c--)
    for(d=10; d>0; d--)
    {
      if(count<10) {m=0; n=0; o=0; p=1;}
      else if(count<100) {m=0; n=0; o=1; p=1;}
      else if(count<1000) {m=0; n=1; o=1; p=1;}
      else if(count<10000) {m=1; n=1; o=1; p=1;}

      *outf=arr[a*m];
      *outk=arr[b*n];
      *outa=arr[c*o];
      *outc=arr[d*p];
      for(i=0; i<1000; i++);
      count--;
      if(count==0) goto end;
    }
  }
  end:
  *outf=0; *outk=0; *outa=0; *outc=0;
}

void loop() {}
```

09 Display the two-digit prime numbers in loop :-

```
void setup() {
  volatile char *dirf, *outf, *dirk, *outk;// *dira,
               *outa, *dirc, *outc;

  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,
                      0x6d,0x7d,0x07,0x7f, 0x6f};

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
  //dira=0x21; *dira=0xff; outa=0x22;
  // dirc=0x27; *dirc=0xff; outc=0x28;

  volatile char a, b, prime, ones, tens;
  volatile long i;

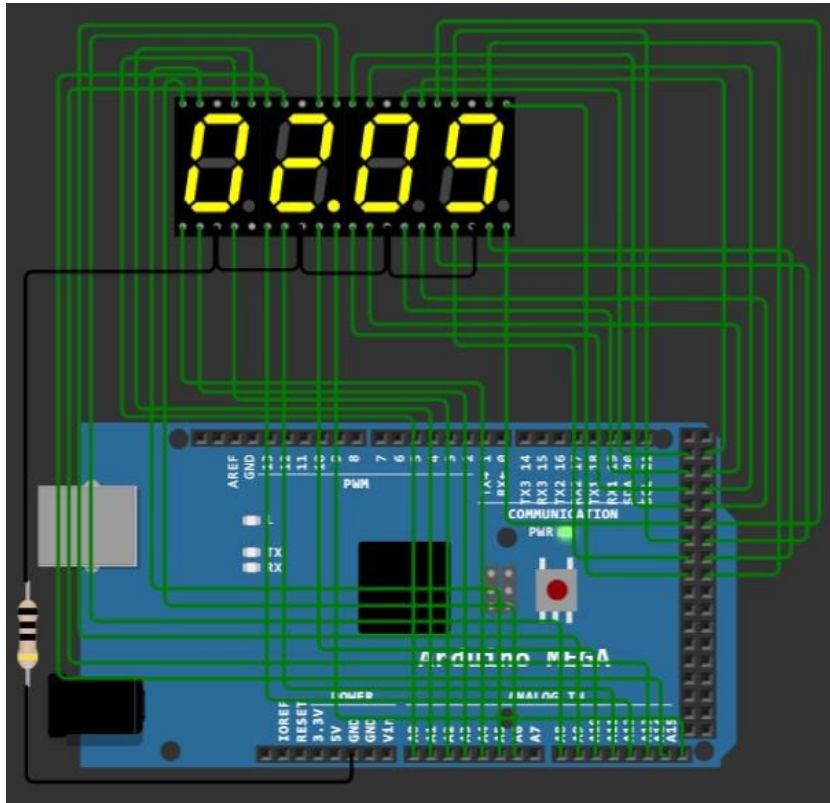
  for(a=10; a<100; a++)
  {
    for(b=2; b<a; b++)
    if((a%b)==0) {prime=0; break;}
    else prime=a;

    if(prime){
      ones=prime%10;
      tens=prime/10;

      *outk=arr[ones];
      *outf=arr[tens];
      for(i=0; i<500000; i++);
    }
  }
  *outf=0;
  *outk=0;
}

void loop() {}
```

10 Display a clock (Min & Sec only) :-



```
void setup() {  
  volatile char *dirf, *outf, *dirk, *outk, *dira, *outa, *dirc, *outc;  
  volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f, 0x6f};  
  dirf=0x30; *dirf=0xff; outf=0x31;  
  dirk=0x107; *dirk=0xff; outk=0x108;  
  dira=0x21; *dira=0xff; outa=0x22;  
  dirc=0x27; *dirc=0xff; outc=0x28;  
  
  volatile char a, b, c, d;  
  
  for(a=0; a<6; a++)  
  for(b=0; b<10; b++)  
  for(c=0; c<6; c++)  
  for(d=0; d<10; d++)  
  {  
    *outf=arr[a];  
    *outk=arr[b] + 128;  
    *outa=arr[c];  
    *outc=arr[d];  
  
    delay(1000);  
  }  
  *outf=0; *outk=0; *outa=0; *outc=0;  
}  
  
void loop() {}
```

ESP – Keypad – Problems Set - 1

01 Press any switch & display that number. Ex: 1 -

> 1, 2 ->2, ... 8 -> 8,9 -> 9,0 -> 0 :-

```
volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
              *dirc, *inc;

volatile char checkKey();
void display(volatile char m);
void init();

void setup() {
    volatile char key;
    init();

while(1){
    key=checkKey();
    display(key);
}

void display(volatile char number){

    volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
                        0x7d,0x07,0x7f, 0x6f, 0x40};

    volatile char a, b, d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
    volatile long i;

if(number){
    for(a=number, b=number; a<(number+11); a++, b++){
        if(b==10) b=0;
        *outk=d4;
        *outf=arr[b];
        for(i=0; i<200000; i++);
    }
}

volatile char checkKey(){

    volatile char row, col, key;
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }

    if(row==0){
        if(col==1) key=1;
        else if(col==2) key=2;
        else if(col==4) key=3;
        else if(col==8) key=20;
    }
}
```

02 Press 4 digits and display that Number.

Ex: 1234,6754, 9870, etc :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
              *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile short number=0;

void setup() {
    volatile char key;
    volatile long i;
    init();

while(1){
    key=checkKey();

    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key);
    }

    display();
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if (number==0) number=key;
    else number = (number*10) + key;

    number%=10000;

    thousand=number/1000;
    hundred=(number%1000)/100;
    tens=(number%100)/10;
    ones=number%10;
}

void display(){

if(flag==1) goto level;
    volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
                        0x7d,0x07,0x7f, 0x6f, 0x40};
    volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
    flag=1;
```



```

else if(row==1){
  if(col==1) key=4;
  else if(col==2) key=5;
  else if(col==4) key=6;
  else if(col==8) key=21;
}
else if(row==2){
  if(col==1) key=7;
  else if(col==2) key=8;
  else if(col==4) key=9;
  else if(col==8) key=22;
}
else if(row==3){
  if(col==1) key=26;
  else if(col==2) key=25;
  else if(col==4) key=24;
  else if(col==8) key=23;
}
else key=0;

return key;
}

```

```

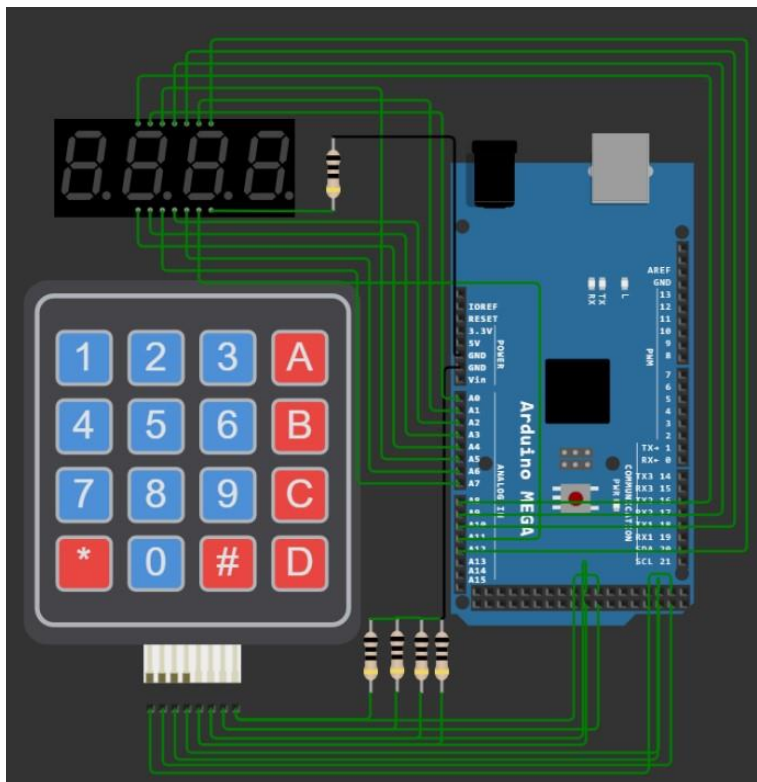
void init(){
  dira=0x21; *dira=0xff; outa=0x22;
  dirc=0x27; *dirc=0; inc=0x26;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
}

```

```

void loop() {}

```



level:

```

*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}

```

```

volatile char checkKey(){
  volatile char row, col, key;

```

```

  for(row=0; row<4; row++){
    *outa= 1 << row;
    if(*inc){ col=*inc; break;}
  }

```

```

  if(row==0){
    if(col==1) key=1;
    else if(col==2) key=2;
    else if(col==4) key=3;
    else if(col==8) key='+';
  }

```

```

  else if(row==1){
    if(col==1) key=4;
    else if(col==2) key=5;
    else if(col==4) key=6;
    else if(col==8) key='-';
  }

```

```

  else if(row==2){
    if(col==1) key=7;
    else if(col==2) key=8;
    else if(col==4) key=9;
    else if(col==8) key='*';
  }

```

```

  else if(row==3){
    if(col==1) key='c';
    else if(col==2) key='0';
    else if(col==4) key='=';
    else if(col==8) key='/';
  }

```

```

  else key=0;

```

```

  return key;
}

```

```

void init(){
  dira=0x21; *dira=0xff; outa=0x22;
  dirc=0x27; *dirc=0; inc=0x26;
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;
}

```

```

void loop() {}

```

03 Perform Single Digit Addition Ex: 6 + 7 = 13, 9 + 4 = 13 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile short number=0, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();

    while(1){
        key=checkKey();

        if(key){
            for(i=0; i<10000; i++) display();
            calculation(key);
        }

        display();
    }
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;}
    else if(key=='='){ num2=number; number=num1+num2;}
    else number=key;

    if(key=='c') number=num1=num2=0;

    number%=100;

    tens=number/10;
    ones=number%10;
}

void display(){
    if(flag==1) goto level;
    volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
                          0x7d,0x07,0x7f, 0x6f, 0x40};
    volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
    flag=1;
```

04 Perform Two Digit Addition Ex: 61 + 72 = 133, 19 + 42 = 61 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile short number=0, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();

    while(1){
        key=checkKey();
        if(key){
            for(i=0; i<10000; i++) display();
            calculation(key);
        }
        display();
    }
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;}
    else if(key=='='){ num2=number;
        number=num1+num2;}
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') number=num1=num2=0;

    if(key=='=') number%=1000;
    else number%=100;

    hundred=number/100;
    tens=(number%100)/10;
    ones=number%10;
}

void display(){
    if(flag==1) goto level;
    volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
                          0x7d,0x07,0x7f, 0x6f, 0x40};
    volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
```

level:

```
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}
```

```
volatile char checkKey(){
    volatile char row, col, key;
```

```
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }
```

```
    if(row==0){
        if(col==1) key=1;
        else if(col==2) key=2;
        else if(col==4) key=3;
        else if(col==8) key='+';
    }
```

```
    else if(row==1){
        if(col==1) key=4;
        else if(col==2) key=5;
        else if(col==4) key=6;
        else if(col==8) key='-';
    }
```

```
    else if(row==2){
        if(col==1) key=7;
        else if(col==2) key=8;
        else if(col==4) key=9;
        else if(col==8) key='*';
    }
```

```
    else if(row==3){
        if(col==1) key='c';
        else if(col==2) key='0';
        else if(col==4) key='=';
        else if(col==8) key='/';
    }
```

```
    else key=0;
```

```
    return key;
```

```
}
```

```
void init(){
```

```
    dira=0x21; *dira=0xff; outa=0x22;
    dirc=0x27; *dirc=0; inc=0x26;
    dirf=0x30; *dirf=0xff; outf=0x31;
    dirk=0x107; *dirk=0xff; outk=0x108;
```

```
}
```

```
void loop() {}
```

```
flag=1;
```

level:

```
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}
```

```
volatile char checkKey(){
    volatile char row, col, key;
```

```
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }
```

```
    if(row==0){
        if(col==1) key=1;
        else if(col==2) key=2;
        else if(col==4) key=3;
        else if(col==8) key='+';
    }
```

```
    else if(row==1){
        if(col==1) key=4;
        else if(col==2) key=5;
        else if(col==4) key=6;
        else if(col==8) key='-';
    }
```

```
    else if(row==2){
        if(col==1) key=7;
        else if(col==2) key=8;
        else if(col==4) key=9;
        else if(col==8) key='*';
    }
```

```
    else if(row==3){
        if(col==1) key='c';
        else if(col==2) key='0';
        else if(col==4) key='=';
        else if(col==8) key='/';
    }
```

```
    else key=0;
```

```
    return key;
```

```
}
```

```
void init(){
```

```
    dira=0x21; *dira=0xff; outa=0x22;
    dirc=0x27; *dirc=0; inc=0x26;
    dirf=0x30; *dirf=0xff; outf=0x31;
    dirk=0x107; *dirk=0xff; outk=0x108;
```

```
}
```

```
void loop() {}
```

05 Perform three Digit Addition. Ex: 161 + 656 = 817
659 + 765 = 1424 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile short number=0, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();

while(1){
    key=checkKey();
    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key);
    }
    display();
} }

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;}
    else if(key=='='){ num2=number; number=num1+num2;}
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') number=num1=num2=0;

    if(key=='=') number%=10000;
    else number%=1000;

    thousand=number/1000;
    hundred=(number%1000)/100;
    tens=(number%100)/10;
    ones=number%10;
}

void display(){

if(flag==1) goto level;
volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
                     0x7d,0x07,0x7f, 0x6f, 0x40};
volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
flag=1;
```

06 Perform any digit Addition. If the total exceeds 9999 then display "EEEE" Ex: 1561 + 2 = 1563, 8765 + 987 = 9752 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile long number=0, temp, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();

while(1){
    key=checkKey();
    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key);
    }
    display();
} }

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;}
    else if(key=='='){ num2=number;
        number=num1+num2;}
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') number=num1=num2=0;
    if(key=='=' &&
        number>9999){thousand=hundred=tens=ones=10;
        goto level;}

    temp=number;
    temp%=10000;

    thousand=temp/1000;
    hundred=(temp%1000)/100;
    tens=(temp%100)/10;
    ones=temp%10;
    level;;
}

void display(){
```

level:

```
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}
```

```
volatile char checkKey(){
    volatile char row, col, key;
```

```
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }
```

```
    if(row==0){
        if(col==1) key=1;
        else if(col==2) key=2;
        else if(col==4) key=3;
        else if(col==8) key='+';
    }
    else if(row==1){
        if(col==1) key=4;
        else if(col==2) key=5;
        else if(col==4) key=6;
        else if(col==8) key='-';
    }
    else if(row==2){
        if(col==1) key=7;
        else if(col==2) key=8;
        else if(col==4) key=9;
        else if(col==8) key='*';
    }
    else if(row==3){
        if(col==1) key='c';
        else if(col==2) key='0';
        else if(col==4) key='=';
        else if(col==8) key='/';
    }
    else key=0;
```

```
    return key;
```

```
}
```

```
void init(){
    dira=0x21; *dira=0xff; outa=0x22;
    dirc=0x27; *dirc=0; inc=0x26;
    dirf=0x30; *dirf=0xff; outf=0x31;
    dirk=0x107; *dirk=0xff; outk=0x108;
}
```

```
void loop() {}
```

```
if(flag==1) goto level;
volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
                    0x7d,0x07,0x7f, 0x6f, 0x79};
volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
flag=1;
```

level:

```
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}
```

```
volatile char checkKey(){
    volatile char row, col, key;
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }
```

```
    if(row==0){
        if(col==1) key=1;
        else if(col==2) key=2;
        else if(col==4) key=3;
        else if(col==8) key='+';
    }
```

```
    else if(row==1){
        if(col==1) key=4;
        else if(col==2) key=5;
        else if(col==4) key=6;
        else if(col==8) key='-';
    }
```

```
    else if(row==2){
        if(col==1) key=7;
        else if(col==2) key=8;
        else if(col==4) key=9;
        else if(col==8) key='*';
    }
```

```
    else if(row==3){
        if(col==1) key='c';
        else if(col==2) key='0';
        else if(col==4) key='=';
        else if(col==8) key='/';
    }
```

```
    else key=0;
    return key;
```

```
}
```

```
void init(){
    dira=0x21; *dira=0xff; outa=0x22;
    dirc=0x27; *dirc=0; inc=0x26;
    dirf=0x30; *dirf=0xff; outf=0x31;
    dirk=0x107; *dirk=0xff; outk=0x108;
}
```

```
void loop() {}
```

07 Perform any digit Subtraction. If the total exceeds 9999 then display "EEEE". Display Negative result also. Ex: 1561 - 2 = 1559, 765 - 987 = -22 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile long number=0, temp, num1, num2, negative;

void setup() {
    volatile char key;
    volatile long i;
    init();
}

while(1){
    key=checkKey();
    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key); }
    display();
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;}
    else if(key=='='){ num2=number; number=num1-num2;}
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') {number=num1=num2=0; negative=0;}
    if(key=='=' &&
    number>9999){thousand=hundred=tens=ones=10; goto
    level;}

    if(number<0){negative=number; number-=1; number =
    ~number;} // For -ve value to +ve value conversion

    temp=number;
    temp%=10000;

    if(negative<0 && number<999) thousand=11;
    else thousand=temp/1000;
        hundred=(temp%1000)/100;
        tens=(temp%100)/10;
        ones=temp%10;
    level;;
```

08 Perform any digit Multiplication. If the total exceeds 9999 then display "EEEE". Ex: 25 x 40 = 1000, 1111 x 8 = 8888 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile short number=0, temp, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();
}

while(1){
    key=checkKey();
    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key);
    }
    display();
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;}
    else if(key=='='){ num2=number;
        number=num1*num2;}
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') number=num1=num2=0;
    if(key=='=' &&
    number>9999){thousand=hundred=tens=ones=10;
    goto level;}

    temp=number;
    temp%=10000;

    thousand=temp/1000;
    hundred=(temp%1000)/100;
    tens=(temp%100)/10;
    ones=temp%10;
    level;;
}

void display(){
```

```

}

void display(){
if(flag==1) goto level;
volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
0x7d,0x07,0x7f, 0x6f, 0x79, 0x40};
volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
flag=1;
level:
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}

volatile char checkKey(){
volatile char row, col, key;
for(row=0; row<4; row++){
*outa= 1 << row;
if(*inc){ col=*inc; break;}
}
if(row==0){
if(col==1) key=1;
else if(col==2) key=2;
else if(col==4) key=3;
else if(col==8) key='+';
}
else if(row==1){
if(col==1) key=4;
else if(col==2) key=5;
else if(col==4) key=6;
else if(col==8) key='-';
}
else if(row==2){
if(col==1) key=7;
else if(col==2) key=8;
else if(col==4) key=9;
else if(col==8) key='*';
}
else if(row==3){
if(col==1) key='c';
else if(col==2) key='0';
else if(col==4) key='=';
else if(col==8) key='/';
}
else key=0;
return key;
}

void init(){
dira=0x21; *dira=0xff; outa=0x22;
dirc=0x27; *dirc=0; inc=0x26;
dirf=0x30; *dirf=0xff; outf=0x31;
dirk=0x107; *dirk=0xff; outk=0x108;
}

void loop() {}

```

```

if(flag==1) goto level;
volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
0x7d,0x07,0x7f, 0x6f, 0x79};
volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
flag=1;

level:
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}

volatile char checkKey(){
volatile char row, col, key;
for(row=0; row<4; row++){
*outa= 1 << row;
if(*inc){ col=*inc; break;}
}

if(row==0){
if(col==1) key=1;
else if(col==2) key=2;
else if(col==4) key=3;
else if(col==8) key='+';
}
else if(row==1){
if(col==1) key=4;
else if(col==2) key=5;
else if(col==4) key=6;
else if(col==8) key='-';
}
else if(row==2){
if(col==1) key=7;
else if(col==2) key=8;
else if(col==4) key=9;
else if(col==8) key='*';
}
else if(row==3){
if(col==1) key='c';
else if(col==2) key='0';
else if(col==4) key='=';
else if(col==8) key='/';
}
else key=0;
return key;
}

void init(){
dira=0x21; *dira=0xff; outa=0x22;
dirc=0x27; *dirc=0; inc=0x26;
dirf=0x30; *dirf=0xff; outf=0x31;
dirk=0x107; *dirk=0xff; outk=0x108;
}

void loop() {}

```

09 Perform any digit Division. Display only Quotient. Ex: 1561 / 6 = 260, 765 / 9 = 85 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile long number=0, temp, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();
}

while(1){
    key=checkKey();
    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key);
    }
    display();
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;
    }
    else if(key=='='){ num2=number; number=num1/num2; }
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') number=num1=num2=0;
    if(key=='=' &&
    number>9999){thousand=hundred=tens=ones=10; goto
    level;}

    temp=number;
    temp%=10000;

    thousand=temp/1000;
    hundred=(temp%1000)/100;
    tens=(temp%100)/10;
    ones=temp%10;
    level;;
}

void display(){
    if(flag==1) goto level;
    volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
```

10 Perform any digit Division. Display only Remainder. Ex: 1561 / 6 = 1, 765 / 9 = 0 :-

```
volatile char checkKey();
void calculation(volatile char key);
void display();
void init();

volatile char *dirf, *outf, *dirk, *outk, *dira, *outa,
               *dirc, *inc;
volatile char ones, tens, hundred, thousand;
volatile char flag=0;
volatile long number=0, temp, num1, num2;

void setup() {
    volatile char key;
    volatile long i;
    init();
}

while(1){
    key=checkKey();
    if(key){
        for(i=0; i<10000; i++) display();
        calculation(key);
    }
    display();
}

void calculation(volatile char key){
    if(key=='0') key=0;

    if(key=='+' || key=='-' || key=='*' || key=='/'){
        num1=number; number=0;
    }
    else if(key=='='){ num2=number;
    number=num1%num2; }
    else if(number==0) number=key;
    else number = (number*10) + key;

    if(key=='c') number=num1=num2=0;
    if(key=='=' &&
    number>9999){thousand=hundred=tens=ones=10;
    goto level;}

    temp=number;
    temp%=10000;

    thousand=temp/1000;
    hundred=(temp%1000)/100;
    tens=(temp%100)/10;
    ones=temp%10;
    level;;
}

void display(){
    if(flag==1) goto level;
```



```
0x7d,0x07,0x7f, 0x6f, 0x79};
volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
flag=1;
```

level:

```
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}
```

```
volatile char checkKey(){
    volatile char row, col, key;
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }
```

```
if(row==0){
    if(col==1) key=1;
    else if(col==2) key=2;
    else if(col==4) key=3;
    else if(col==8) key='+';
}
```

```
else if(row==1){
    if(col==1) key=4;
    else if(col==2) key=5;
    else if(col==4) key=6;
    else if(col==8) key='-';
}
```

```
else if(row==2){
    if(col==1) key=7;
    else if(col==2) key=8;
    else if(col==4) key=9;
    else if(col==8) key='*';
}
```

```
else if(row==3){
    if(col==1) key='c';
    else if(col==2) key='0';
    else if(col==4) key='=';
    else if(col==8) key='/';
}
```

```
else key=0;
return key;
```

```
}
void init(){
    dira=0x21; *dira=0xff; outa=0x22;
    dirc=0x27; *dirc=0; inc=0x26;
    dirf=0x30; *dirf=0xff; outf=0x31;
    dirk=0x107; *dirk=0xff; outk=0x108;
}
```

```
void loop() {}
```

```
volatile char arr[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,
0x7d,0x07,0x7f, 0x6f, 0x79};
volatile char d4=~1, d3=~2, d2=~4, d1=~8, clr=~16;
flag=1;
```

level:

```
*outk=d4; *outf=arr[thousand]; *outk=d4; *outf=0;
*outk=d3; *outf=arr[hundred]; *outk=d3; *outf=0;
*outk=d2; *outf=arr[tens]; *outk=d2; *outf=0;
*outk=d1; *outf=arr[ones]; *outk=d1; *outf=0;
}
```

```
volatile char checkKey(){
    volatile char row, col, key;
    for(row=0; row<4; row++){
        *outa= 1 << row;
        if(*inc){ col=*inc; break;}
    }
```

```
if(row==0){
    if(col==1) key=1;
    else if(col==2) key=2;
    else if(col==4) key=3;
    else if(col==8) key='+';
}
```

```
else if(row==1){
    if(col==1) key=4;
    else if(col==2) key=5;
    else if(col==4) key=6;
    else if(col==8) key='-';
}
```

```
else if(row==2){
    if(col==1) key=7;
    else if(col==2) key=8;
    else if(col==4) key=9;
    else if(col==8) key='*';
}
```

```
else if(row==3){
    if(col==1) key='c';
    else if(col==2) key='0';
    else if(col==4) key='=';
    else if(col==8) key='/';
}
```

```
else key=0;
return key;
```

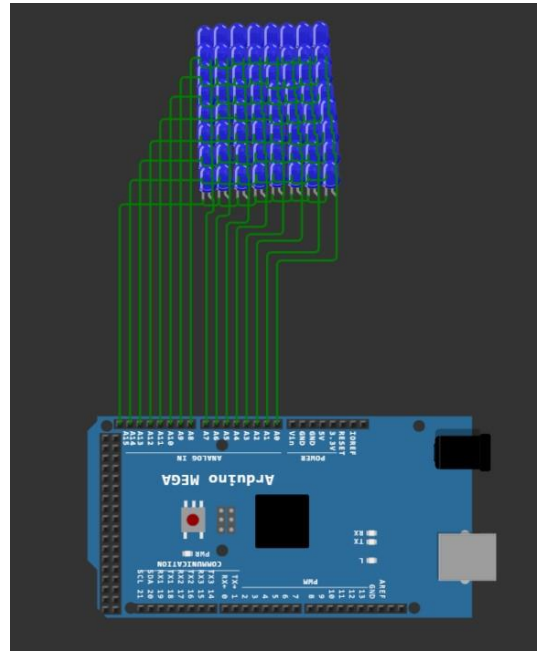
```
}
void init(){
    dira=0x21; *dira=0xff; outa=0x22;
    dirc=0x27; *dirc=0; inc=0x26;
    dirf=0x30; *dirf=0xff; outf=0x31;
    dirk=0x107; *dirk=0xff; outk=0x108;
}
```

```
void loop() {}
```

ESP – Dot-matrix – Problems Set - 1

01 Display Right most Top LED :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=1;  
  *outk=~1;  
}  
void loop() {}
```



02 Display Left most Top LED :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=128;  
  *outk=~1;  
}  
void loop() {}
```

03 Display Right most bottom LED :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=1;  
  *outk=~128;  
}  
void loop() {}
```

04 Display Left most bottom LED :-

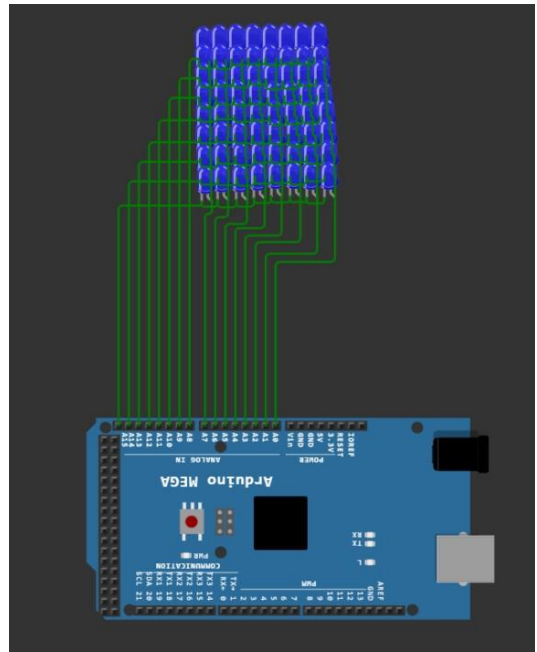
```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=128;  
  *outk=~128;  
}  
void loop() {}
```

05 Display Topmost Row all LEDs :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=0xff;  
  *outk=~1;  
}  
void loop() {}
```

06 Display Bottom most Row all LEDs :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=0xff;  
  *outk=~128;  
}  
void loop() {}
```



07 Display Left most Column all LEDs :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=128;  
  *outk=0;  
}  
void loop() {}
```

08 Display Right most Column all LEDs :-

```
void setup() {  
  
  volatile char *dirf, *dirk, *outf, *outk;  
  volatile long a;  
  char i, count=0;  
  
  dirf=0x30; *dirf=0xff;  
  dirk=0x107; *dirk=0xff;  
  outf=0x31; outk=0x108;  
  
  *outf=1;  
  *outk=0;  
}  
void loop() {}
```

09 Display diagonal LEDs connecting Left top LED to Right bottom LED :-

```
void setup() {  
  volatile char *dirf, *dirk, *outf, *outk, row, col;  
  
  dirf=0x30; *dirf=0xff; outf=0x31;  
  dirk=0x107; *dirk=0xff; outk=0x108;  
  
  while(1){  
    for(row=0; row<9; row++)  
      for(col=0; col<9; col++)  
        if(row==col){  
          *outf= 1 << row; *outk= ~( 128 >> col);  
          *outf=0; *outk=0xff;  
        }  
      }  
  }  
  void loop() {}
```

10 Display diagonal LEDs connecting Right top LED to Left bottom LED :-

```
void setup() {  
  volatile char *dirf, *dirk, *outf, *outk, row, col;  
  
  dirf=0x30; *dirf=0xff; outf=0x31;  
  dirk=0x107; *dirk=0xff; outk=0x108;  
  
  while(1){  
    for(row=0; row<9; row++)  
      for(col=0; col<9; col++)  
        if(row==col){  
          *outf= 1 << row; *outk= ~( 1 << col);  
          *outf=0; *outk=0xff;  
        }  
      }  
  }  
  void loop() {}
```

ESP – Dot-matrix – Problems Set – 2

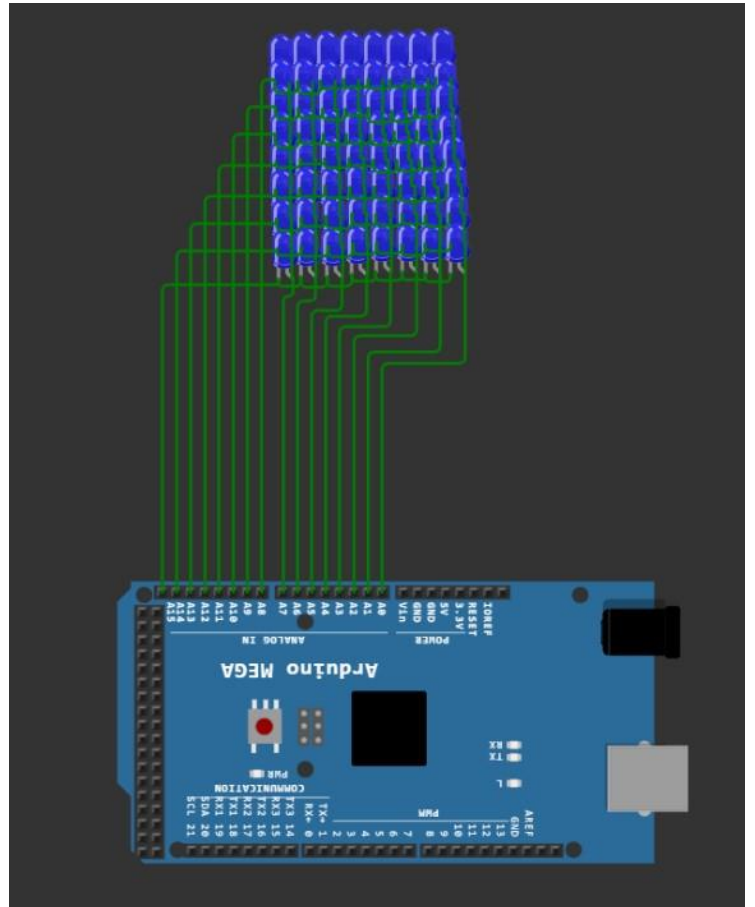
01 Display Character A :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00000000,
   0b00011000,
   0b00100100,
   0b01111110,
   0b10000001,
   0b00000000,
   0b00000000,
   0b00000000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
  void loop() {}
```



02 Display Character B :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b11111100,
   0b10000010,
   0b10000010,
   0b11111100,
   0b10000010,
   0b10000010,
   0b11111100,
   0b00000000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
  void loop() {}
```

03 Display Number 1 :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00011000,
   0b00111000,
   0b00011000,
   0b00011000,
   0b00011000,
   0b00011000,
   0b00011000,
   0b00111100};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
  void loop() {}
```

04 Display Number 2 :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00111110,
   0b01111110,
   0b01100110,
   0b00001110,
   0b00011100,
   0b00111000,
   0b01111111,
   0b01111111};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
}
void loop() {}
```

05 Display Character a :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00011000,
   0b00100100,
   0b00000100,
   0b00011100,
   0b00100100,
   0b00100100,
   0b00011010,
   0b00000000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
}
void loop() {}
```

06 Display Character q :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00111111,
   0b01111111,
   0b01100011,
   0b01111111,
   0b00111111,
   0b00000011,
   0b00000011,
   0b00000011};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
}
void loop() {}
```

07 Display Symbol \$:-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00010000,
   0b00111000,
   0b01010100,
   0b01010000,
   0b00111000,
   0b00010100,
   0b01010100,
   0b00111000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
}
void loop() {}
```

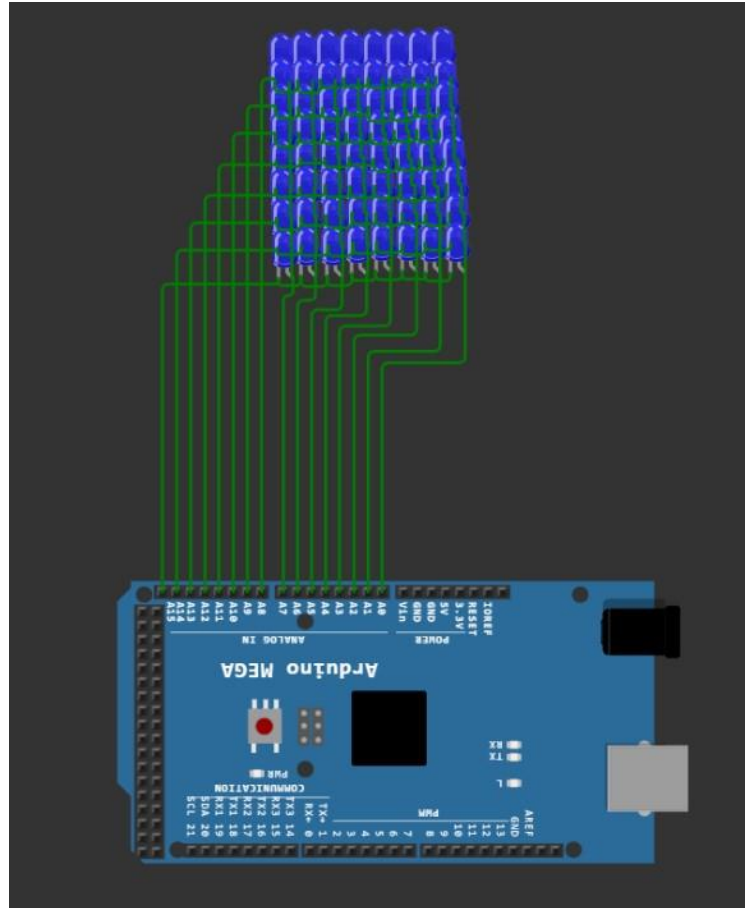
08 Display Symbol # :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00000000,
   0b00010100,
   0b01111110,
   0b00101000,
   0b11111100,
   0b01010000,
   0b00000000,
   0b00000000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
  void loop() {}
```



09 Display Symbol α :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00000000,
   0b00000000,
   0b00011010,
   0b00100100,
   0b00100100,
   0b00011010,
   0b00000000,
   0b00000000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
  void loop() {}
```

10 Display Symbol Ω :-

```
void setup() {
  volatile char i, *dirf, *dirk, *outf, *outk;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char num[] =
  {0b00000000,
   0b00000000,
   0b00111000,
   0b01000100,
   0b01000100,
   0b00101000,
   0b11101110,
   0b00000000};

  while(1){
    for(i=0; i<8; i++){
      *outk= ~(1 << i); *outf = num[i];
      *outk=0xff; *outf=0;
    }
  }
  void loop() {}
```

ESP – Dot-matrix – Problems Set – 3

01 Display Characters A to E one by one in one second Interval :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a;
  char i, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char abcde[][8] = {
    {0b00000000, 0b00011000, 0b00100100,
     0b01111110, 0b10000001, 0b00000000,
     0b00000000, 0b00000000}, // A

    {0b01111000, 0b01000100, 0b01000100,
     0b01111000, 0b01000100, 0b01000100,
     0b01111000, 0b00000000}, // B

    {0b00000000, 0b00111000, 0b01000100,
     0b01000000, 0b01000000, 0b01000100,
     0b00111000, 0b00000000}, // C

    {0b00000000, 0b01111000, 0b01000100,
     0b01000100, 0b01000100, 0b01000100,
     0b01111000, 0b00000000}, // D

    {0b00000000, 0b01111100, 0b01000000,
     0b01111100, 0b01000000, 0b01111100,
     0b00000000, 0b00000000}}; // E

  while(1){
    for(a=0; a<20000; a++){
      for(i=0; i<8; i++){
        {
          *outk= ~(1 << i);
          *outf = ecen[count][i];
          *outk=0xff;
          *outf=0;
        }
      }
    }

    for(a=0; a<45000; a++){
      count++;
      if(count==5) count=0;

    }
  }
  void loop() {}
```

02 Display Characters a to e one by one in one second Interval :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a;
  char i, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char ecen[][8] = {
    {0b00011000, 0b00100100, 0b000000100,
     0b00011100, 0b00100100, 0b00100100,
     0b00011010, 0b00000000}, // a

    {0b00100000, 0b00100000, 0b00100000,
     0b00111000, 0b00100100, 0b00100100,
     0b00011000, 0b00000000}, // b

    {0b00000000, 0b00011000, 0b00100100,
     0b00100000, 0b00100000, 0b00100100,
     0b00011000, 0b00000000}, // c

    {0b000000100, 0b000000100, 0b000000100,
     0b00011100, 0b00100100, 0b00100100,
     0b00011000, 0b00000000}, //d

    {0b00000000, 0b00011000, 0b00100100,
     0b00111100, 0b00100000, 0b00011000,
     0b00000000, 0b00000000}}; //e

  while(1){
    for(a=0; a<20000; a++){
      for(i=0; i<8; i++){
        {
          *outk= ~(1 << i);
          *outf = ecen[count][i];
          *outk=0xff;
          *outf=0;
        }
      }
    }

    for(a=0; a<45000; a++){
      count++;
      if(count==5) count=0;

    }
  }
  void loop() {}
```

03 Display Numbers 0 to 4 one by one in one second Interval :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a;
  char i, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char ecen[][8] = {
    {0b00010000, 0b00110000, 0b00010000,
     0b00010000, 0b00010000, 0b00010000,
     0b00010000, 0b00111000}, // 1

    {0b00111100, 0b01000010, 0b01000100,
     0b00001000, 0b00010000, 0b00100000,
     0b01111110, 0b00000000}, // 2

    {0b00111110, 0b00000100, 0b00001000,
     0b00011100, 0b00000010, 0b01000010,
     0b00111100, 0b00000000}, // 3

    {0b00000000, 0b00010100, 0b00100100,
     0b01000100, 0b01111110, 0b00000100,
     0b00000100, 0b00000000}, // 4

    {0b00000000, 0b00111100, 0b00100000,
     0b00111000, 0b00000100, 0b00100100,
     0b00011000, 0b00000000}};

  while(1){
    for(a=0; a<20000; a++){
      for(i=0; i<8; i++){
        {
          *outk= ~(1 << i);
          *outf = ecen[count][i];
          *outk=0xff;
          *outf=0;
        }
      }
    }

    for(a=0; a<45000; a++){
      count++;
      if(count==5) count=0;

    }
  }
  void loop() {}
```

04 Display Welcome string characters one by one in one second Interval :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a;
  char i, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char ecen[][8] = {
    {0b11000011, 0b11000011, 0b11000011,
     0b11000011, 0b11011011, 0b11111111,
     0b11100111, 0b11000011}, // W

    {0b00000000, 0b00000000, 0b00000000,
     0b00111100, 0b01100110, 0b01111110,
     0b01100000, 0b00111100}, // e

    {0b00011000, 0b00011000, 0b00011000,
     0b00011000, 0b00011000, 0b00011000,
     0b00011000, 0b00001100}, // l

    {0b00000000, 0b00000000, 0b00000000,
     0b00111100, 0b01101100, 0b01100000,
     0b01100000, 0b00111100}, // c

    {0b00000000, 0b00000000, 0b00000000,
     0b00111100, 0b01100110, 0b01100110,
     0b01100110, 0b00111100}, // o

    {0b00000000, 0b00000000, 0b00000000,
     0b01110110, 0b11011011, 0b11011011,
     0b11011011, 0b11011011}, // m

    {0b00000000, 0b00000000, 0b00000000,
     0b00111100, 0b01100110, 0b01111110,
     0b01100000, 0b00111100}}; // e

  while(1){
    for(a=0; a<20000; a++){
      for(i=0; i<8; i++){
        {
          *outk= ~(1 << i);
          *outf = ecen[count][i];
          *outk=0xff;
          *outf=0;
        } }
      }
    }
    for(a=0; a<45000; a++){
      count++;
      if(count==7) count=0;
    }
  }
  void loop() {}
```


05 Display revolving text “Welcome” Right to Left :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a, b;
  char i, j, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char arr[] = {
    0, 0, 0, 0, 0, 0, 0, 0, 0,
    0b11111111, 0b11111111, 0b01100000,
    0b00110000, 0b00110000, 0b01100000,
    0b11111111, 0b11111111, // W
    0b00000000, 0b01110000, 0b11111100,
    0b10101000, 0b10101000, 0b10111000,
    0b00110000, 0b00000000, // e
    //0b00000000, 0b00000000, 0b00000000,
    0b01111111, 0b11111111, 0b10000000,
    //0b00000000, 0b00000000, // l
    0b00000000, 0b01110000, 0b11111100,
    0b10001000, 0b10011000, 0b10011000,
    //0b00000000, 0b00000000, // c
    0b00000000, 0b01110000, 0b11111100,
    0b10001000, 0b10001000, 0b11111100,
    0b01110000, 0b00000000, // o
    0b11110000, 0b11111100, 0b00001000,
    0b11111000, 0b11110000, 0b00001000,
    0b11111000, 0b11110000, // m
    0b00000000, 0b01110000, 0b11111100,
    0b10101000, 0b10101000, 0b10111000,
    0b00110000, 0b00000000, // e
    0, 0, 0, 0, 0, 0, 0, 0,
  };

  while(1){
    for(i=0; i<57; i++){
      for(a=0; a<8000; a++){
        for(j=0; j<8; j++){
          *outf= 128 >> j;
          *outk= ~(arr[i+j]);
          *outk=0xff;
          *outf=0;
        }
      }
      for(b=0; b<32000; b++){
      }
    }
  }
  void loop() {}
```

06 Display revolving text “Welcome” Left to Right :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a, b;
  char i, j, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char arr[] = {
    0, 0, 0, 0, 0, 0, 0, 0, 0,
    0b00000000, 0b01110000, 0b11111100,
    0b10101000, 0b10101000, 0b10111000,
    0b00110000, 0b00000000, // e
    0b11110000, 0b11111100, 0b00001000,
    0b11111000, 0b11110000, 0b00001000,
    0b11111000, 0b11110000, // m
    0b00000000, 0b01110000, 0b11111100,
    0b10001000, 0b10001000, 0b11111100,
    0b01110000, // 0b00000000, // o
    0b00000000, 0b01110000, 0b11111100,
    0b10001000, 0b10011000, 0b10011000,
    0b00000000, //0b00000000, // c
    //0b00000000, 0b00000000, 0b00000000,
    0b01111111, 0b11111111, 0b10000000,
    //0b00000000, 0b00000000, // l
    0b00000000, 0b01110000, 0b11111100,
    0b10101000, 0b10101000, 0b10111000,
    0b00110000, 0b00000000, // e
    0b11111111, 0b11111111, 0b01100000,
    0b00110000, 0b00110000, 0b01100000,
    0b11111111, 0b11111111, // W
    0, 0, 0, 0, 0, 0, 0, 0,
  };

  while(1){
    for(i=56; i>=0; i--){
      for(a=0; a<8000; a++){
        for(j=0; j<8; j++){
          *outf= 128 >> j;
          *outk= ~(arr[i+j]);
          *outk=0xff;
          *outf=0;
        }
      }
      for(b=0; b<32000; b++){
      }
    }
  }
  void loop() {}
```

07 Display revolving text “Welcome” Top to Bottom :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a, b;
  char i, j, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char arr[] = {
    0, 0, 0, 0, 0, 0, 0, 0,
    0b11000011, 0b11100111, 0b11111111,
    0b11011011, 0b11000011, 0b11000011,
    0b11000011, 0b11000011, 0, // W
    0b00111100, 0b01100000, 0b01111110,
    0b01100110, 0b00111100, 0b00000000,
    // 0b00000000, 0b00000000, //e
    0b00001100, 0b00011000, 0b00011000,
    0b00011000, 0b00011000, 0b00011000,
    0b00011000, 0b00011000, 0, // l
    0b00111100, 0b01100000, 0b01100000,
    0b01101100, 0b00111100, 0b00000000,
    // 0b00000000, 0b00000000, //c
    0b00111100, 0b01100110, 0b01100110,
    0b01100110, 0b00111100, 0b00000000,
    // 0b00000000, 0b00000000, //o
    0b11011011, 0b11011011, 0b11011011,
    0b11011011, 0b01110110, 0b00000000,
    // 0b00000000, 0b00000000, //m
    0b00111100, 0b01100000, 0b01111110,
    0b01100110, 0b00111100, 0b00000000,
    //0b00000000, 0b00000000, //e
    0, 0, 0, 0, 0, 0, 0, 0,
  };

  while(1){
    for(i=0; i<58; i++){
      for(a=0; a<8000; a++){
        for(j=0; j<8; j++){
          *outk= ~(128 >> j);
          *outf= arr[i+j];
          *outk=0xff;
          *outf=0;
        }
      }
      for(b=0; b<32000; b++){
      }
    }
  }
  void loop() {}
```

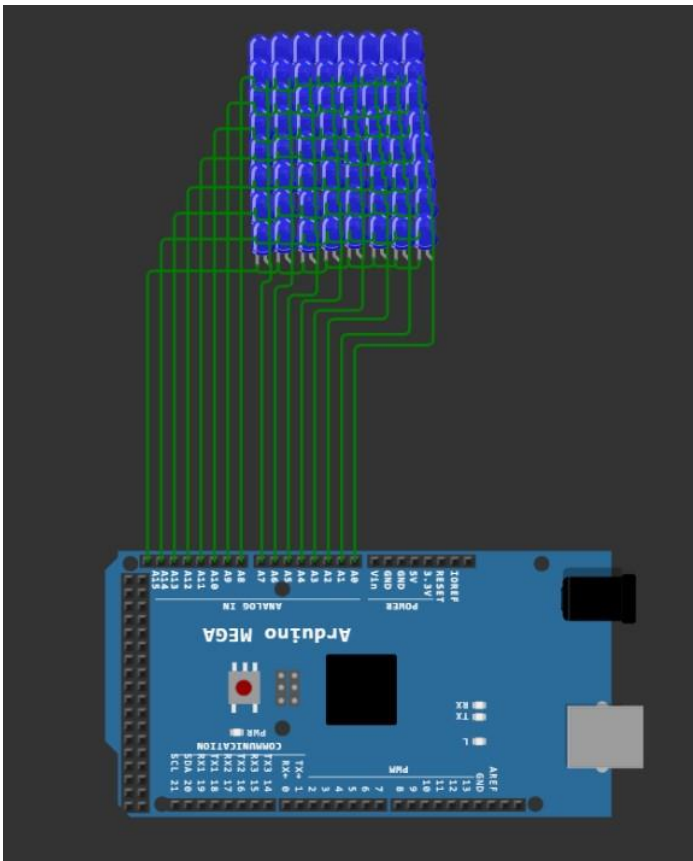
08 Display revolving text “Welcome” Bottom to Top :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a, b;
  char i, j, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char arr[] = {
    0, 0, 0, 0, 0, 0, 0, 0,
    0b11000011, 0b11000011, 0b11000011,
    0b11000011, 0b11011011, 0b11111111,
    0b11100111, 0b11000011, // W
    0b00000000, // 0b00000000, 0b00000000,
    0b00111100, 0b01100110, 0b01111110,
    0b01100000, 0b00111100, 0, //e
    0b00011000, 0b00011000, 0b00011000,
    0b00011000, 0b00011000, 0b00011000,
    0b00011000, 0b00001100, // l
    0b00000000, // 0b00000000, 0b00000000,
    0b00111100, 0b01101100, 0b01100000,
    0b01100000, 0b00111100, //c
    0b00000000, // 0b00000000, 0b00000000,
    0b00111100, 0b01100110, 0b01100110,
    0b01100110, 0b00111100, //o
    0b00000000, // 0b00000000, 0b00000000,
    0b01110110, 0b11011011, 0b11011011,
    0b11011011, 0b11011011, // m
    0b00000000, // 0b00000000, 0b00000000,
    0b00111100, 0b01100110, 0b01111110,
    0b01100000, 0b00111100, //e
    0, 0, 0, 0, 0, 0, 0, 0
  };

  while(1){
    for(i=0; i<57; i++){
      for(a=0; a<8000; a++){
        for(j=0; j<8; j++){
          *outk= ~(1 << j);
          *outf= arr[i+j];
          *outk=0xff;
          *outf=0;
        }
      }
      for(b=0; b<32000; b++){
      }
    }
  }
  void loop() {}
```



09 Display “Welcome” by odd position entering from Left and Even Position entering from Right :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a, b;
  char i, j, count=0;

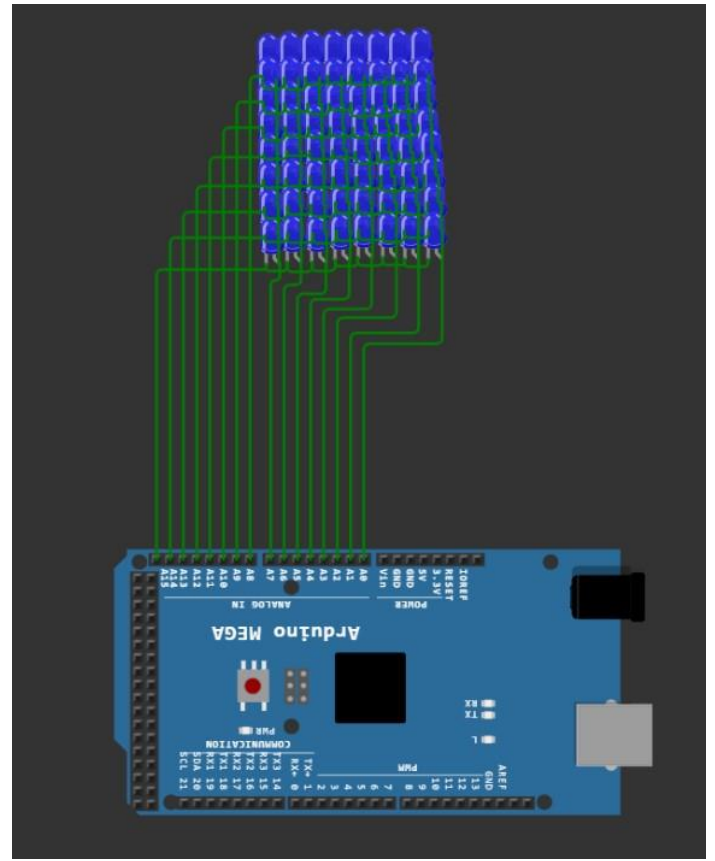
  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char arr[][24] = {

    {0, 0, 0, 0, 0, 0, 0, 0, 0,
     0b11111111, 0b11111111, 0b01100000,
     0b00110000, 0b00110000, 0b01100000,
     0b11111111, 0b11111111,
     0, 0, 0, 0, 0, 0, 0, 0}, // W

    {0, 0, 0, 0, 0, 0, 0, 0, 0,
     0b00000000, 0b01110000, 0b11111000,
     0b10101000, 0b10101000, 0b10111000,
     0b00110000, 0b00000000,
     0, 0, 0, 0, 0, 0, 0, 0}, // e

    {0, 0, 0, 0, 0, 0, 0, 0, 0,
     0b00001100, 0b00011000, 0b00011000,
     0b00011000, 0b00011000, 0b00011000,
     0b00011000, 0b00011000, // l
     0, 0, 0, 0, 0, 0, 0, 0}, // l
```



10 Display “Welcome” by odd position entering from Top and Even Position entering from Bottom :-

```
void setup(){
  volatile char *dirf, *dirk, *outf, *outk;
  volatile long a, b;
  char i, j, count=0;

  dirf=0x30; *dirf=0xff; outf=0x31;
  dirk=0x107; *dirk=0xff; outk=0x108;

  volatile char arr[][24] = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0,
     0b11000011, 0b11100111, 0b11111111,
     0b11011011, 0b11000011, 0b11000011,
     0b11000011, 0b11000011, // W
     0, 0, 0, 0, 0, 0, 0, 0},

    {0, 0, 0, 0, 0, 0, 0, 0, 0,
     0b00111100, 0b01100000, 0b01111110,
     0b01100110, 0b00111100, 0b00000000,
     0b00000000, 0b00000000, //e
     0, 0, 0, 0, 0, 0, 0, 0},

    {0, 0, 0, 0, 0, 0, 0, 0, 0,
     0b00001100, 0b00011000, 0b00011000,
     0b00011000, 0b00011000, 0b00011000,
     0b00011000, 0b00011000, // l
     0, 0, 0, 0, 0, 0, 0, 0},
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b00000000, 0b01110000, 0b11111000,
0b10001000, 0b10011000, 0b10011000,
0b00000000, 0b00000000,
0, 0, 0, 0, 0, 0, 0, 0}, // c
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b00000000, 0b01110000, 0b11111000,
0b10001000, 0b10001000, 0b11111000,
0b01110000, 0b00000000,
0, 0, 0, 0, 0, 0, 0, 0}, // o
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b11110000, 0b11111000, 0b00001000,
0b11111000, 0b11110000, 0b00001000,
0b11111000, 0b11110000,
0, 0, 0, 0, 0, 0, 0, 0}, // m
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b00000000, 0b01110000, 0b11111000,
0b10101000, 0b10101000, 0b10111000,
0b00110000, 0b00000000,
0, 0, 0, 0, 0, 0, 0, 0}}; // e
```

```
while(1){
    loop:
    for(i=16; i>=0; i--){
        for(a=0; a<8000; a++){
            for(j=0; j<8; j++){
                *outf= 128 >> j;
                *outk= ~(arr[count][i+j]);
                *outk=0xff;
                *outf=0;
            }
        }
        for(b=0; b<32000; b++){
        }
        count++;
        if(count>6){count=0; goto loop;}
```

```
    for(i=0; i<17; i++){
        for(a=0; a<8000; a++){
            for(j=0; j<8; j++){
                *outf= 128 >> j;
                *outk= ~(arr[count][i+j]);
                *outk=0xff;
                *outf=0;
            }
        }
        for(b=0; b<32000; b++){
        }
        count++;
    }
}
void loop() {}
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b00111100, 0b01100000, 0b01100000,
0b01101100, 0b00111100, 0b00000000,
0b00000000, 0b00000000, //c
0, 0, 0, 0, 0, 0, 0, 0},
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b00111100, 0b01100110, 0b01100110,
0b01100110, 0b00111100, 0b00000000,
0b00000000, 0b00000000, //o
0, 0, 0, 0, 0, 0, 0, 0},
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b11011011, 0b11011011, 0b11011011,
0b11011011, 0b01110110, 0b00000000,
0b00000000, 0b00000000, // m
0, 0, 0, 0, 0, 0, 0, 0},
```

```
{0, 0, 0, 0, 0, 0, 0, 0, 0,
0b00111100, 0b01100000, 0b01111110,
0b01100110, 0b00111100, 0b00000000,
0b00000000, 0b00000000, //e
0, 0, 0, 0, 0, 0, 0, 0}};
```

```
while(1){
    loop:
    for(i=0; i<17; i++){
        for(a=0; a<8000; a++){
            for(j=0; j<8; j++){
                *outk= ~(128 >> j);
                *outf= arr[count][i+j];
                *outk=0xff;
                *outf=0;
            }
        }
        for(b=0; b<32000; b++){
        }
        count++;
        if(count>6){count=0; goto loop;}
```

```
    for(i=16; i>=0; i--){
        for(a=0; a<8000; a++){
            for(j=0; j<8; j++){
                *outk= ~(128 >> j);
                *outf= arr[count][i+j];
                *outk=0xff;
                *outf=0;
            }
        }
        for(b=0; b<32000; b++){
        }
        count++;
    }
}
void loop() {}
```

